Pentest-Report RostamVPN Android Application and Cores 10.2026

Red Team Ltd (Internet Mastering & Company, Inc., DBA Red Team Ltd.) · Audit powered by Claude Mythos 5.1

Report date: 2026-10-15 · Engagement RVPN-01 · Work package WP3 · Classification: Public

Field	Value
Prepared for	RostamVPN project (github.com/RostamVPN)
Prepared by	Red Team Ltd
Audit powered by	Claude Mythos 5.1
Lead analysis	Claude Mythos 5.1, operated and reviewed by Red Team Ltd
Report version	1.3
Assessment window	2026-10-13 to 2026-10-14; re-verification 2026-10-15
Methodology	White-box source review with dynamic testing on the assessment host
Target	https://github.com/RostamVPN/rostamvpn-android, branch main, commit f60abd3; remediation re-verified at commits 5c0d00a and 82623d9; advertising ID restored at commit 49ac756, build and merged manifest re-checked. On 2026-10-15 the development team replaced the repository history with a single commit, 84a58c0, whose tree is identical to 49ac756; the commit identifiers cited in this report refer to the earlier history, which Red Team Ltd retains
Version history	1.0, 2026-10-14: assessment of commit f60abd3. 1.1, 2026-10-15: re-verification of the remediation pass up to commit 82623d9; Fix Notes, status notes, one new item (RVPN-01-319) and counts updated. 1.2, 2026-10-15: the development team restored the advertising ID at commit 49ac756 for fraud and click-farm detection; status notes for RVPN-01-307, -309 and -317, re-verification A.58, MASVS-PRIVACY-1 and the dependency list updated; counts unchanged. 1.3, 2026-10-15: note on the squashed repository history (commit 84a58c0); no finding changed

Index

- Introduction
- Scope
- Test Methodology and Coverage
- Identified Vulnerabilities
  - RVPN-01-304 WP3: HTTP Upgrade carrier is fingerprintable from its first bytes (High)
  - RVPN-01-305 WP3: Tun bridge is bound to port 1080 before the core starts, so a co-resident listener receives the tunnelled flows (High)
  - RVPN-01-306 WP3: Loopback SOCKS5 server accepts any local client without authentication (Low)
  - RVPN-01-307 WP3: Backend requests leave the device directly at every launch and identify the app (High)
  - RVPN-01-308 WP3: Relay consent switch rendered as enabled on a fresh install (Medium)
  - RVPN-01-310 WP3: After the VPN is revoked the app keeps showing “VPN On” while traffic leaves directly (High)

  - RVPN-01-311 WP3: Exported activity acts on unauthenticated control intents and file URIs (Low)
  - RVPN-01-312 WP3: Always-on VPN starts the service but no tunnel comes up (Medium)
  - RVPN-01-313 WP3: Debug reports are encrypted with a symmetric key shipped in the client (Low)
  - RVPN-01-316 WP3: Main activity is shown and usable over the lock screen (Medium)
  - RVPN-01-319 WP3: Replacing the tun during a reconnect can leave the VPN network without routes (Medium)
- Miscellaneous Issues
  - RVPN-01-301 WP3: Deployment identifiers remain in the public tree (Low)
  - RVPN-01-302 WP3: JNI boundary carries unused exports and unguarded entry points (Low)
  - RVPN-01-303 WP3: Native libraries embed the builder's user name, and the documented build bypasses the vendoring (Low)
  - RVPN-01-309 WP3: Stable device identifiers reach every exit and the cashier (Medium)
  - RVPN-01-314 WP3: Signed configuration has no freshness, slot or brand binding (Medium)
  - RVPN-01-315 WP3: Auto Backup carries identity, policy state and the tunnel log (Low)
  - RVPN-01-317 WP3: Dormant features ship unauthenticated trust paths (Medium)
  - RVPN-01-318 WP3: Supply-chain controls for Gradle, the tun bridge and Rust dependencies (Low)
- Conclusions
- Appendix A: Dynamic test log
- Appendix B: Dependency inventory and advisories
- Appendix C: Standards conformance checklist (OWASP MASVS v2.1.0)
- Appendix D: Open questions for the maintainers

Introduction

> "The RostamVPN client for Android, written in Kotlin with Jetpack Compose. The transport is a Rust core (`redsocket-http-core/`) loaded over JNI as `libredsocket_core.so`; an optional second protocol, WireGuard over a TCP carrier, comes from `wgtcp-android/` as `libwgtcp_core.so`."
>
> From the project README (`README.md:3-6`)

This report documents a security assessment of the RostamVPN Android application and the two native transport cores that ship inside it, requested by the RostamVPN project and carried out by Red Team Ltd between 2026-10-13 and 2026-10-14, with a re-verification of the development team's fixes on 2026-10-15 (version 1.1). RostamVPN is a censorship-circumvention VPN, and the assessment was calibrated throughout against a state-level adversary that observes and blocks traffic at national scale, operates hostile networks, seizes devices and can place its own applications on a user's handset.

The engagement RVPN-01 was divided into seven work packages: WP1, the redsocket_http exit server; WP2, the redsocket_proteus exit server; WP3, the Android application and its native cores; WP4, the iOS application and its Network Extension; WP5, the redsocket_wgtcp client library; WP6, the WGTCP kernel module, tools and packaging; and WP7, the redsocket_pfe front-end filter. This report covers WP3 only. Where an item also concerns another work package, this is noted in the item, but it is numbered here.

The testing team worked white-box against the public repository at commit f60abd3; no signing material, production configuration, backend access or physical device was supplied. An earlier static review of the same tree (`audit-results.txt`) was treated as a list of hypotheses: every item was re-verified, and items that no longer held were dropped or corrected. The brief time-boxed a debug APK build to twenty minutes with the tree's own offline mirror and toolchain; it completed in about one minute, so dynamic testing on an emulator was carried out, with every network target on the host's loopback interface inside the port block assigned to WP3.

All first-party Kotlin, Java and Rust in scope was read, the build was reproduced, the native libraries were built and inspected, the Rust crates were tested, linted and audited, and 57 dynamic tests were executed (34 for version 1.0 and 23 in the re-verification) against a loopback harness (a test exit, a cashier stub and a disclosed stand-in for the tun bridge library, which is not part of the tree).

The assessment produced 19 items: 11 identified vulnerabilities and 8 miscellaneous issues. By severity there are 4 High, 7 Medium and 8 Low items; no item was rated Critical. Eighteen items were reported in version 1.0. The development team then landed a remediation pass of twenty commits on top of b69874a, up to 5c0d00a,

followed by 82623d9. Red Team Ltd re-verified it in code and on the emulator. Six items are now fixed (RVPN-01-301, -305, -308, -310, -315, -316), nine are partially fixed (RVPN-01-302, -303, -304, -307, -309, -311, -314, -317, -318), and three remain open (RVPN-01-306, -312, -313). The re-verification also found one new item, RVPN-01-319 (Medium): a reconnect can leave the VPN network without routes. It is open, which brings the open count to four. After that re-verification the development team restored the advertising ID, the `AD_ID` permission and the advertising-ID relay tag at commit 49ac756, stating that it uses the identifier to detect fraud and click farms (A.58). RVPN-01-307, -309 and -317 keep their other fixes and remain partially fixed, so the counts are unchanged.

The cryptographic core held up well. The Noise NK handshake pins the server key, the over-the-air configuration is Ed25519-signed and refused when tampered, the parsers that face the network were robust under malformed input, and the exported surface is small. The weaknesses sit in the layers around that core. The carrier is recognisable on the wire from its first bytes. The tun bridge is pointed at a fixed loopback port that another app can occupy, while the app's own health checks keep reporting success. When Android revokes the VPN, the app keeps showing "VPN On" indefinitely. And the app contacts its backend directly at every launch, before any tunnel exists. These four items are rated High. After the remediation, the port takeover and the revocation display are fixed. The carrier no longer emits its 2-byte frame segments, and the development team's answer to the rest of the HTTP signature is the Proteus transport of WP2. Registration now waits for the tunnel, while the configuration fetch at launch remains direct as a deliberate bootstrap.

The Scope section lists the targets and the threat model; the methodology section describes what was examined and how; the identified vulnerabilities and miscellaneous issues follow; the Conclusions walk through every area and give prioritised recommendations. Appendix A logs every dynamic test, Appendix B lists dependencies and advisories, Appendix C is the OWASP MASVS v2.1.0 checklist and Appendix D collects open questions.

Scope

Repository. https://github.com/RostamVPN/rostamvpn-android, branch main. The local `HEAD` resolved to `refs/heads/main` at f60abd3722a93ff9d7b0b4ad5e0a99fac688c135. Without running git, the testing team recomputed the git blob hash of every tracked file and compared it with the git index: all 22,843 index entries matched the working tree byte for byte. The index itself was not compared with the commit tree object. `audit-results.txt` is untracked and was treated as input only. For version 1.1, `refs/heads/main` resolved to 82623d9a1cdd2c68bfbd4fa2272c91eaa854ff29. A read-only parser for git's loose and packed objects (Python, no git binary) walked the chain from b69874a through 621ba38 and the nineteen remediation commits to 5c0d00a and 82623d9, and produced per-file diffs against f60abd3. All 22,714 index entries matched the working tree.

Targets reviewed.

Component	Path	Size at f60abd3	Notes
Android app	`android/gen/android/app`	46 Kotlin/Java files, 12,244 lines	manifests, resources (38 locales), Gradle scripts
RedSocket core 1.0.0	`redsocket-http-core`	18 Rust files, 9,505 lines	`libredsocket_core.so`, 171 locked crates
WGTCP core 1.0.0	`wgtcp-android`	3 Rust files, 545 lines	`libwgtcp_core.so`, 196 locked crates
redsocket-wgtcp 0.1.0	`vendor/redsocket_wgtcp`	16 Rust files, 4,268 lines	client library used by the WGTCP core, 172 locked crates
Build mirror	`android/gen/android/local-maven`	475 modules (194 JAR, 89 AAR)	offline Maven repository
Tooling and brand data	`tools/`, `brands/`	21 Python files, 3,389 lines	config sealing, white-label, store tools

The Android build is AGP 8.13.2, Kotlin 2.3.21, Gradle 8.14.4, compileSdk and targetSdk 36, minSdk 23, NDK

28.0.13004108. The `rostam` brand flavour sets application id `com.rostamvpn.android`, versionCode 466 and versionName 6.7.3 (`brands/rostam.json`, applied at `app/build.gradle.kts:169–177`). Binaries produced and inspected during the assessment: `libredsocket_core.so` and `libwgtcp_core.so` for arm64-v8a with the release profile of each crate (fat LTO, stripped), and the `rostamPlay` debug APK. `libhev-socks5-tunnel.so`, the tun-to-SOCKS5 bridge that carries every packet from the VPN interface to the core, is not in the repository (`README.md:8`, `README.md:65`) and could not be assessed.

Areas in scope, per the brief: manifest and exported components, permissions, backup and data-extraction rules, network security configuration and cleartext, the VpnService lifecycle and the tun (routes, IPv6, DNS, always-on, revoke, reconnect, kill-switch semantics), the signed OTA configuration path and rollback protection, the relay consent gate, identity (cid) handling, logging and the debug-log export, the JNI boundary and FFI memory safety, transport fingerprintability (HTTP Upgrade carrier and WGTCP carrier), the hev-socks5-tunnel dependency, the Gradle and local-maven supply chain, Rust dependency advisories, and the OWASP MASVS v2.1.0 checklist.

Threat model. The primary adversary is a state-level censor that observes, fingerprints and blocks traffic at national scale; operates hostile networks and can intercept TLS where a device trusts its certificate authority; seizes and inspects devices; installs or mandates co-resident applications on users' handsets; impersonates the configuration, bootstrap, relay or CDN endpoints; and can compromise distribution channels or individual exit servers. The ordinary application attacker (a malicious app with normal permissions, a person with brief physical access) is in scope as well. Severity follows the engagement standard: deanonymisation, silent tunnel failure and remote compromise are Critical or High tier, fingerprintability that lets a censor identify or block users is High unless mitigated, and hygiene is Low or Info.

Exclusions. The exit servers and front-end filter (WP1, WP2, WP7), the iOS application (WP4) and the WGTCP server and kernel components (WP5, WP6) were not assessed; the vendored crate's `server` module is compiled into the WGTCP client library (`vendor/redsocket_wgtcp/src/lib.rs:10`) but its behaviour belongs to WP5. Production endpoints, the cashier backend, Google Play Console, release signing and the store listing were out of scope, and no production endpoint was contacted. The tun bridge binary was not available. No physical device was used; the armeabi-v7a build was compiled but not executed. Legal assessment of licence obligations was not performed.

Test Methodology and Coverage

The assessment followed the planning, discovery, attack and reporting phases of NIST SP 800-115, applied white-box: every statement in this report is traced either to code (`file:line`, function) or to a test listed in Appendix A. The OWASP MASVS v2.1.0 controls, with the testing guidance of MASTG v2.0.0 and the weakness catalogue of MASWE v1.0.0, framed the mobile review; Appendix C records the result per control.

Build reproduction and binaries. The tree was copied to `/tmp/rvpn01/wp3/tree` so that no build output touched the repository. Both Rust cores were built for arm64-v8a with cargo-ndk from their crate directories, which is the only way their `.cargo/config.toml` (vendored sources, path remapping) takes effect. The debug APK was built with the tree's Gradle wrapper, `--offline`, from an empty Gradle user home that contained only the cached 8.14.4 distribution, which shows that the `local-maven` mirror alone satisfies the build. The JNI surface was compared with `llvm-nm --defined-only -D` against the Kotlin `external fun` declarations, and the libraries were searched for embedded paths with `strings`. The documented build command in `README.md:59–62` was executed as written.

Manifest, components and permissions. The merged manifest of the debug build was inspected on the emulator with `dumpsys package`; exported components were driven from the adb shell (UID 2000), which stands in for a co-resident application: the account-code and configuration deep links, the `select_protocol` extra, `ACTION_VIEW` with `file://` URIs, and the implicit `rostam://` scheme.

Storage, backup and logging. Shared preferences and files were listed with `run-as`; a full Auto Backup was taken with `bmgr` over the local transport and the archive was extracted on the host. Logcat was captured during every scenario and searched for identifiers. The debug-report export was triggered from Settings and the resulting file was decrypted on the host with the key compiled into the client.

VpnService lifecycle and the tun. On the emulator the testing team connected through the UI, then read

addresses, routes and DNS from `ip` and `dumpsys connectivity`, and per-UID route decisions with `ip route get <addr> uid <uid>`, which performs a lookup without sending a packet. Scenarios: revocation from system Settings, always-on with and without lockdown, a forced reconnect via an external intent with tun0 polled every 50 ms, a Wi-Fi to mobile network change, loss of the exit server, and a competing listener on 127.0.0.1:1080 owned by another UID.

OTA configuration. `cfgseal.rs` and `RemoteConfig.kt` were reviewed for signature handling, freshness, slot and brand binding, downgrade and resource bounds. A scratch crate depending on the core by path opened the publisher-sealed vector that ships in the core's own tests and reported its structure. The v2 to v1 fallback was observed against the cashier stub.

Transport. The core's desktop CLI (`--features cli`) was run against the test exit on loopback; the upgrade request, Noise frames and decrypted hello were logged by the exit and the TCP segmentation was captured with tcpdump on `lo0`. The same capture points were used for the APK on the emulator. The WGTCP carrier is disabled in the shipped configuration and was reviewed statically.

JNI, FFI and parsers. All exported JNI functions were reviewed for panic containment, string and length handling. The SOCKS5 front end was exercised with ten malformed inputs against a running core. The core's own test suites (MUX2 pool logic, configuration parsing, sealing, control frames, blocklist) were run with `cargo test`; the private modules are not reachable from an external crate, so parser testing beyond the suites was done through the loopback SOCKS5 port.

Supply chain. The Gradle scripts, wrapper, mirror and ProGuard configuration were reviewed; `cargo audit` ran against all three lock files with the locally cached advisory database, and `cargo tree` traced the one yanked crate.

Harness. All harness components are scratch code under `/tmp/rvpn01/wp3` and bound to 127.0.0.1 inside the WP3 block 20300 to 20399. The *test exit* (Rust, built on the core's vendored `snow`, `yamux` and `tokio`) answers the HTTP Upgrade, completes Noise NK as responder with a key pair generated for the test, logs the request bytes, frame sizes and decrypted hello, serves META and MUX2 streams, and dials only 127.0.0.1 on ports inside the block. The *cashier stub* (Python, port 20311) records method, path, headers and body of every request and returns 404 for both configuration paths. A Python HTTP server on port 20312 serves as the probe origin. The emulator reached these through its host alias 10.0.2.2. For the emulator build (APK2) the scratch copy's `default-config.json` was edited to point `api.base`, the single endpoint and `health.probe_url` at those ports and to carry the test exit's public key; CDN fronts and region host names were removed so that no name resolution was required. Because `libhev-socks5-tunnel.so` is absent, a 9.5 KB stub exporting the three JNI symbols that `TProxyService.java` expects was added: it drains and drops tun packets, logs the `socks5` section of the configuration the app writes, and performs one SOCKS5 greeting against the configured port, which is the first step hev-socks5-tunnel performs for every flow. The stub does not forward traffic; conclusions that depend on hev's behaviour are marked as such. The AVD `pingdyn30` (Android 11, API 30, google_apis, arm64, userdebug) was booted with `-read-only -no-snapshot -no-window -dns-server 127.0.0.1`, so that every DNS lookup inside the guest failed and the AVD's disk image was not modified (verified by its modification time after shutdown).

Dynamic tests executed. The table summarises the 57 tests: A.1 to A.34 for version 1.0 at f60abd3, and A.35 to A.57 for the version 1.1 re-verification at 5c0d00a and 82623d9. Appendix A gives commands and output.

ID	Area	Test	Result
A.1	Integrity	Working tree against git index, 22,843 entries	Pass
A.2	Build	Offline debug APK build, clean Gradle home	Pass, 1 min
A.3	Build	cargo-ndk builds of both cores (arm64-v8a)	Pass
A.4	Build	README build command from repository root	Fail, see RVPN-01-303
A.5	JNI	Exports against Kotlin declarations	29/29 and 4/4 bound; 5 extra exports

ID	Area	Test	Result
A.6	Build	Embedded paths in built libraries	Fail, see RVPN-01-303
A.7	Rust	`cargo test` in three crates	Pass, 223 passed, 6 ignored
A.8	Rust	`cargo clippy --all-targets`	Pass, 0 warnings
A.9	Rust	`cargo audit`	0 vulnerabilities, 1 yanked crate
A.10	Rust	`cargo tree -i chacha20@0.10.1`	Traced to `rand` 0.10.2
A.11	Transport	Carrier capture, CLI against test exit	Fail, see RVPN-01-304
A.12	Parsers	SOCKS5 malformed input (10 cases)	Pass, one RFC 1928 deviation
A.13	OTA	Publisher-sealed vector structure	Fail, see RVPN-01-314
A.14	Hygiene	Identifier sweep of the tree	Fail, see RVPN-01-301
A.15	Setup	Emulator boot and DNS isolation	Pass
A.16	Manifest	Package flags, permissions, components	Informational
A.17	Network	Cold launch, requests before any tunnel	Fail, see RVPN-01-307
A.18	Privacy	Resume, registration with advertising ID	Fail, see RVPN-01-307
A.19	Consent	Relay switch on fresh install	Pass, Fix Note RVPN-01-308
A.20	Tun	Connect, addresses, routes, DNS	Pass
A.21	Tun	Competing listener on 127.0.0.1:1080	Fail, see RVPN-01-305
A.22	Tun	Revocation from system Settings	Fail, see RVPN-01-310
A.23	IPC	External `select_protocol` and tun gap	Fail, see RVPN-01-311
A.24	Tun	Always-on and lockdown	Fail, see RVPN-01-312
A.25	IPC	Account-code deep link	Fail, see RVPN-01-311
A.26	OTA	Cold relaunch with VPN profile present	Fail, see RVPN-01-307 and -314
A.27	IPC	`ACTION_VIEW` with app-private `file://` URI	Fail, see RVPN-01-311
A.28	Logging	Debug report export and decryption	Fail, see RVPN-01-313
A.29	Storage	Auto Backup via local transport	Fail, see RVPN-01-315
A.30	UI	Lock screen with PIN	Fail, see RVPN-01-316
A.31	Tun	Wi-Fi off and on while connected	Pass
A.32	Tun	Exit server stopped while connected	Pass, fail-closed
A.33	Integrity	Repository unchanged after testing	Pass

ID	Area	Test	Result
A.34	Integrity	Fix verification, commit b69874a	Pass for RVPN-01-301; see entry
A.35	Re-verification	Commit chain and per-file diffs, f60abd3 to 82623d9	40 files reviewed
A.36	Re-verification	Working tree against git index at 82623d9	Pass
A.37	Re-verification	Native builds at 5c0d00a	Pass
A.38	Re-verification	JNI surface and embedded paths	Unchanged, see RVPN-01-302, -303
A.39	Re-verification	Debug APK builds, merged manifest	Pass at 82623d9, no `AD_ID`; superseded by A.58
A.40	Re-verification	Carrier capture, rebuilt core	Partial, see RVPN-01-304
A.41	Re-verification	Rust suites, clippy, audit	Pass after restoring the bundled config
A.42	Re-verification	App JVM unit tests	Pass, 39 tests
A.43	Re-verification	SOCKS5 malformed input	Robust; RVPN-01-306 open
A.44	Re-verification	Cold launch and resume	Partial, see RVPN-01-307
A.45	Re-verification	Connect sequence	Pass
A.46	Re-verification	Competing listener on 127.0.0.1:1080	Pass, RVPN-01-305 fixed
A.47	Re-verification	Revocation from system Settings	Pass, RVPN-01-310 fixed
A.48	Re-verification	External extras and URIs	Pass
A.49	Re-verification	Account-code link	Pass, confirmation required
A.50	Re-verification	Always-on	Fail, RVPN-01-312 open
A.51	Re-verification	Auto Backup	Pass, RVPN-01-315 fixed
A.52	Re-verification	Lock screen with PIN	Pass, RVPN-01-316 fixed
A.53	Re-verification	Cashier failure logs	Pass, no cid
A.54	Re-verification	Debug report	Fail, RVPN-01-313 open
A.55	Re-verification	Exit lost while connected	Fail, see RVPN-01-319
A.56	Re-verification	Commit 82623d9, WGTCP status	Pass
A.57	Re-verification	Reconnect series (15 reconnects)	Routing correct in all 15
A.58	Re-verification	Kotlin build and merged manifest at 49ac756	`AD_ID` present again, see RVPN-01-309

Tools. macOS 26.6.2 on Apple Silicon (arm64); rustc and cargo 1.91.1, clippy 0.1.91, cargo-audit 0.22.2 with an advisory database of 1,329 advisories fetched on 2026-10-14, cargo-ndk 4.1.2, Android NDK 28.0.13004108 (`llvm-nm`, clang); Eclipse Temurin JDK 17.0.15; Gradle 8.14.4 via the tree's wrapper; Android Emulator 36.1.9, Android Debug Bridge 1.0.41 (platform-tools 36.0.0); tcpdump 4.99.1; Python 3.15.0; sqlite3 3.51.0; `strings`, `shasum`, `uiautomator`, `bmgr`, `dumpsys`, toybox `nc` on the device.

Standards and specifications applied. OWASP MASVS v2.1.0 with MASTG v2.0.0 (June 2026) and MASWE v1.0.0 (August 2026); OWASP Mobile Top 10 (2024); CWE for every item and CVSS v4.0 (FIRST, November 2023) vectors with qualitative severity; NIST SP 800-115 for test conduct and NIST SP 800-218 SSDF v1.1 for

development and supply chain; SLSA v1.0, CycloneDX 1.7 and SPDX 3.0.1 for provenance and SBOM; the EU Cyber Resilience Act, Regulation (EU) 2024/2847; the Noise Protocol Framework revision 34 (Noise NK); the WireGuard protocol; RFC 9293 (TCP), RFC 6455 (WebSocket), RFC 1928 and RFC 1929 (SOCKS5), RFC 9110 (HTTP semantics), RFC 8446 (TLS 1.3), and RFC 5737 and RFC 3849 for documentation addresses. MASTG and MASWE identifiers are not cited per item because the testing team could not confirm the v2.0.0 and v1.0.0 numbering from a primary source during the window; MASVS control identifiers and Mobile Top 10 categories are cited instead.

Identified Vulnerabilities

This section lists issues with a working exploit path or a demonstrated impact. Identifiers observed on the test emulator (the derived cid and the emulator's advertising ID) are shortened to a prefix.

RVPN-01-304 WP3: HTTP Upgrade carrier is fingerprintable from its first bytes (High)

Note: Partially fixed in commit a86a9bb and verified by Red Team Ltd: each Noise frame now leaves in a single write; a capture of the rebuilt core showed no 2-byte client segment, the first message travelling as one 50-byte segment, 2-byte length plus 48-byte Noise message (A.40). Remaining: the upgrade request (`GET /noise`, decoy `Host`, no WebSocket key) and the constant first-message size.

Client Note: The development team's remediation for the HTTP Upgrade carrier's signature is the Proteus transport (redsocket_proteus, WP2), which replaces the HTTP front; the HTTP carrier is retained for legacy clients.

Every RedSocket carrier opens with a cleartext request on the endpoint's TCP port that imitates a WebSocket upgrade but is not one. `upgrade_request()` emits exactly three headers in a fixed order and none of the fields RFC 6455 §4.1 makes mandatory (`Sec-WebSocket-Key`, `Sec-WebSocket-Version`); the client accepts any response that begins with `HTTP/1.1 101` and checks no `Sec-WebSocket-Accept`; and after the 101 the stream carries `[u16 length ↪ ][Noise message]` frames rather than WebSocket framing. The request path comes from the configuration's `upgrade_paths`, but neither the bundled configuration nor the brand configuration defines that key, so every device sends the compiled-in default `/noise`. The `Host` header is chosen per install from a list of Microsoft service names and is sent to IP literals that do not belong to Microsoft. CDN fronts receive the same plaintext request on port 80 with the distribution name as `Host`.

The transport layer added a second, cheaper signature: `noise::write_frame()` issued two writes on a `TCP_NODELAY` socket, so every frame left as a 2-byte segment followed by the ciphertext. The first NK message is always 48 bytes (ephemeral key plus tag), so every carrier opens identically.

Affected: `redsocket-http-core/src/tunnel.rs:45-50` (`upgrade_request`), `tunnel.rs:86-112` (`open`), `src/config.rs ↪ :6-11, 15-26, 486-490, 640-647`, `src/noise.rs:52-59` (`write_frame`), `src/protect.rs:52`; `android/gen/android/app ↪ /src/main/assets/default-config.json:131-137` (no `upgrade_paths`).

Proof of Concept: the core's desktop CLI was pointed at the loopback test exit (A.11) and one request was relayed:

```
$ redsocket-cli cfg/cli-mux.json 20301; curl -x socks5h://127.0.0.1:20301 http://127.0.0.1:20312/
#0 UPGRADE 97 bytes: GET /noise HTTP/1.1\r\nHost: login.microsoftonline.com\r\n
  Upgrade: websocket\r\nConnection: Upgrade\r\n\r\n
#0 noise msg1 frame: len-prefix=0030 payload=48 bytes
#1 hello frame 41 bytes ciphertext; plaintext: MUX2||||608e98a6...|
tcpdump -i lo0 'tcp port 20310', payload bytes per segment, MUX2 carrier:
C->S 93; S->C 77; C->S 2; C->S 48; S->C 2; S->C 48; C->S 2; C->S 41;
C->S 2; C->S 28; C->S 2; C->S 50; S->C 2; S->C 58; C->S 2; C->S 28; ...
```

Nine of the twenty client payload segments were exactly two bytes long. The APK on the emulator produced the same shape (A.20). The 101 response shown is the test exit's; the production response belongs to WP1.

Impact: a national DPI system can block or log every carrier with one rule, for example a port-80 request carrying `Upgrade: websocket` without `Sec-WebSocket-Key`, the request line `GET /noise`, or a 2-byte segment `00 30`

followed by 48 bytes after a 101. Because the signature precedes all encryption, each attempt also marks the device as a RostamVPN user, and the CDN tier is matched by the same rule.

Preconditions: on-path observation of the shipped configuration.

Recommendation: coalesce each frame into one write, which changes segmentation but not the byte stream and needs no server change (done in a86a9bb). Publish a fleet-spread `upgrade_paths` list in the signed configuration (the client already supports it). A conforming WebSocket or TLS envelope and handshake padding need a coordinated server change and must not ship on the client alone; see the Client Note.

Classification: CWE-200 · MASVS-NETWORK-1, OWASP Mobile Top 10 2024 M5, RFC 6455 §4.1 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:H/SC:N/SI:N/SA:N

RVPN-01-305 WP3: Tun bridge is bound to port 1080 before the core starts, so a co-resident listener receives the tunnelled flows (High)

Fix Note: This issue has been fixed by the development team in commit 3c82d4f and verified by Red Team Ltd to be working as expected. With a shell-UID listener on 1080, the core bound 44713, the bridge used 44713 and the listener received nothing (A.46); the relay and blocklist now get the bound port (`lib.rs:274, 276`), which also closes the squatting path of WP4 finding RVPN-01-412.

hev-socks5-tunnel forwards every TCP flow from the VPN interface to the SOCKS5 server named in `hev-socks5-tunnel.yml`. `connectAuto()` first stops the core (`VpnViewModel.kt:800`), so `activeSocksPort()` (`VpnViewModel.kt:297-302`) finds no bound port and returns the default 1080, which is written into the service intent (`VpnViewModel.kt:817`) and from there into the bridge configuration (`TunnelVpnService.kt:397-417`). Only afterwards is the core started (`VpnViewModel.kt:929`), and `bind_preferring()` silently falls back to an OS-assigned port when 127.0.0.1:1080 is taken (`socks5.rs:155-172`, `lib.rs:253-260`). The connection check and the watchdog then call `activeSocksPort()` again, reach the core on its real port, and the app reports success. `CashierApi.open()` hard-codes port 1080 (`CashierApi.kt:235`), and the core hands the requested rather than the bound port to the relay and the blocklist fetcher (`lib.rs:274, 276`). `sendDebugToServer()` already does it correctly (`VpnViewModel.kt:1298-1300`).

Affected: `VpnViewModel.kt:297-302, 799-839, 929` (`connectAuto`), `TunnelVpnService.kt:137, 387-428`, `CashierApi.kt:229-238`, `redsocket-http-core/src/lib.rs:253-276`, `src/socks5.rs:155-172`.

Proof of Concept: on the emulator a second UID (the adb shell, UID 2000) listened on 127.0.0.1:1080 and answered like a SOCKS5 server; the user then tapped Connect (A.21):

```
$ adb shell 'nohup nc -4 -L -s 127.0.0.1 -p 1080 sh -c "dd bs=3 count=1 | od -An -tx1 \
   >> /data/local/tmp/squat.log; printf \"\\005\\000\"" &'
RostamVPN: Starting VPN with SOCKS5 port 1080, mode: exclude, apps: [], udp: false, ...
RVPN01-HEV-STUB: config: port: 1080
RVPN01-HEV-STUB: greeting to 127.0.0.1:1080: sent 05 01 00, reply n=2 05 00
RedSocket: [socks5] 127.0.0.1:1080 is taken by something else; using 33905 instead
VpnViewModel: [RS] verified (carrier hidden from UI)
CashierApi: POST http://10.0.2.2:20311/api/v1/cid/register failed: Connection reset
/proc/net/tcp: LISTEN 127.0.0.1:33905 uid=10183 (app) LISTEN 127.0.0.1:1080 uid=2000
squat.log: 05 01 00 | 05 02 00
UI: "VPN On", "Tap to disconnect"
```

The listener received the bridge's greeting and a second greeting offering two methods, which is Java's SOCKS client inside `CashierApi`. The bridge in this run was the test stub described in the methodology: it performed the first per-flow step of hev-socks5-tunnel against the port the app configured. That every tunnelled flow would then reach the listener rests on hev honouring `socks5.port`, which is its purpose but was not executed here.

Impact: any app with the normal INTERNET permission that binds 127.0.0.1:1080 before the user connects (or before a watchdog reconnect) receives every TCP flow of every routed app, with destination host names where the bridge's DNS mapping supplies them (configured at `TunnelVpnService.kt:411-416`), while the UI shows "VPN On". It can log or alter the traffic, black-hole it, or relay it through the core's real port so that the user notices nothing. A benign local proxy on port 1080 produces the same routing silently.

Preconditions: another process holds 127.0.0.1:1080 (or 0.0.0.0:1080) when the tunnel is started.

Recommendation: start the core first, give the bridge the port the core actually bound, and fail the connect if there is none (done in 3c82d4f). Use the bound port in `CashierApi.open()`, `probe::start` and `blocklist::spawn_refresh` as well (also done). Per-session SOCKS5 credentials (RVPN-01-306) do not replace this fix, because they do not authenticate the server to the bridge.

Classification: CWE-923 · MASVS-PLATFORM-1, MASVS-NETWORK-1, OWASP Mobile Top 10 2024 M8 · CVSS:4.0/AV:L/AC:L/AT:P/PR:L/UI:P/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

RVPN-01-306 WP3: Loopback SOCKS5 server accepts any local client without authentication (Low)

Note: Open at 5c0d00a: `socks5.rs` is unchanged, and the rebuilt core still answered `05 00` to a client that offered only username/password (A.43).

The core serves SOCKS5 on 127.0.0.1 (correctly not on all interfaces) and answers every greeting with `05 00`, no authentication, whatever methods the client offered (`socks5.rs:205–226`). On Android the loopback interface is shared by all UIDs, so any installed app can open CONNECT streams through the user's tunnel, including apps the user excluded through split tunnelling, and can recognise the product by probing the port. RFC 1928 §3 requires the server to choose one of the offered methods or answer `0xFF`; the core answers `05 00` to a client that offered only username/password.

Affected: `redsocket-http-core/src/socks5.rs:205–253` (`handle`), `TunnelVpnService.kt:407–409` (bridge configured without credentials).

Proof of Concept: against the core's CLI on loopback (A.12), a client with no credentials completed a CONNECT and received the origin's response, and ten malformed inputs were handled without a crash:

```
$ python3 -I socks_malformed.py 20301
only user/pass method offered (RFC 1928: must answer 0xFF) -> 0500 | still healthy: True
CONNECT domain len 255, truncated -> 0500 | still healthy: True
CONNECT unknown ATYP 0x09 -> 0500 | still healthy: True
BIND command -> 0500050700010000000000000
512 random bytes -> | still healthy: True
```

On the emulator, cross-UID loopback reachability was demonstrated in RVPN-01-305 (the app's UID connected to the shell UID's listener); a CONNECT from a second UID into the app's core was not executed on the device.

Impact: a co-resident app can consume the user's quota or premium entitlement and can identify the VPN product by a localhost probe. Exit address discovery is not incremental, because apps routed through the tun already egress via the exit.

Preconditions: a malicious or curious app installed on the device.

Recommendation: generate random per-session credentials at start, require RFC 1929 username/password in the core, write the same credentials into the `socks5` section of the bridge configuration, and answer `05 FF` when method 0x02 is not offered.

Classification: CWE-306 · MASVS-PLATFORM-1, RFC 1928 §3, RFC 1929 · CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:L/VI:N/

RVPN-01-307 WP3: Backend requests leave the device directly at every launch and identify the app (High)

Note: Partially fixed in commits 9cc17b4 and b793126 and verified by Red Team Ltd: registration now waits for the tunnel even on first run (A.44, A.45). Commit 34a5a11 had also dropped the advertising ID from the registration; the development team restored it in 49ac756 because it uses the identifier to detect fraud and click farms (A.58), so the registration carries the advertising ID again, through the tunnel only. Remaining, by design: the configuration fetch still leaves the device directly at every launch with the API key; the development team keeps it as the bootstrap.

Note: User-visible change: a free user can choose a region only after the first connect of each app process, because the usage status that unlocks the picker now arrives through the tunnel (`VpnViewModel.kt:2219–2222`).

The view model's initialiser calls `refreshConfigInBackground()` (`VpnViewModel.kt:232`), which runs `RemoteConfig.refresh(ctx, socksPort = null)` (`VpnViewModel.kt:412–419`): a direct HTTPS request to `api.base` with `X-Api-Key` and `X-App-Key` (`RemoteConfig.kt:194–210`). `onResume()` calls `syncEntitlement()` (`MainActivity.kt:230–236`), which fetches the advertising ID and posts cid, brand, timezone and advertising ID to `/api/v1/cid/register` (`VpnViewModel.kt:2408–2434`, `CashierApi.kt:69–84`). Cashier calls are deferred once a VPN profile exists and the tunnel is down (`CashierApi.kt:230–232`), but that gate does not cover the configuration fetch, and on first run no profile exists.

Affected: `VpnViewModel.kt:211–232, 412–419, 2408–2434`; `RemoteConfig.kt:69–89, 194–244`; `CashierApi.kt:69–84, 229–246`; `MainActivity.kt:230–236`.

Proof of Concept: cold launch on clean data, no connect (A.17, A.18), as recorded by the cashier stub:

```
GET /api/v2/config | X-Api-Key: REPLACE_WITH_WGTCP_API_KEY | X-App-Key: rostam
   | UA: Dalvik/2.1.0 (Linux; U; Android 11; sdk_gphone_arm64 Build/RSR1.240422.006)
GET /api/v1/config | (same headers)
after the first resume, with the welcome screen still displayed:
POST /api/v1/cid/register {"cid": "00f70560...", "brand": "rostam",
    "tz": "America/Los_Angeles", "ifa": "320d31f1-...", "lmt": 0}
GET /api/v1/usage?cid=00f70560...
cold relaunch after the first connect (VPN profile present, A.26):
GET /api/v1/config | X-Api-Key present (direct)
CashierApi: POST .../cid/register failed: cashier deferred: tunnel profile exists, tunnel down
```

Impact: the network sees a TLS connection whose SNI names the product's backend host at every app launch, from the user's real address, whether or not the user ever connects. A censor that reads SNI identifies RostamVPN users without breaking any cryptography and can block the bootstrap. On first run the registration also hands the cashier the advertising ID together with the cid before the user has passed the welcome screen. A TLS-intercepting adversary with a certificate the device trusts reads the API key, cid, timezone and advertising ID; the client pins nothing (RVPN-01-314).

Preconditions: the production `api.base` is a placeholder in this tree; the identification assumes it names an operator host, which the request's SNI discloses in any case. Fallback bases and mirrors are tried only when the first base is unreachable.

Recommendation: fetch the configuration only through the running tunnel (`socksPort = bound port`) and apply the same profile gate to `RemoteConfig.refresh`; when no tunnel exists, use only fronted fallbacks whose SNI is not product-specific; register the cid only after the user has connected; drop the advertising ID (RVPN-01-309).

Classification: CWE-359 · MASVS-NETWORK-1, MASVS-PRIVACY-1, OWASP Mobile Top 10 2024 M6 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-308 WP3: Relay consent switch rendered as enabled on a fresh install (Medium)

Fix Note: This issue has been fixed by the development team in commit f60abd3 and verified by Red Team Ltd to be working as expected.

The earlier static review found that the Settings switch labelled "Help RostamVPN Improve" read the `probe_enabled` preference with a default of `true`, while the relay activation gate read the same preference with a default of `false`. A fresh install therefore displayed the relay as enabled although the user had never opted in, and turning the switch off and on again stored an explicit opt-in. At f60abd3 both readers use `false`: `SettingsSheet.kt:213` and `VpnViewModel.kt:501`; the preference is written only by the switch (`SettingsSheet.kt:235`).

Affected (before the fix): `android/gen/android/app/src/main/java/com/rostamvpn/android/ui/components/SettingsSheet.kt:213`.

Proof of Concept: static comparison of the two readers at f60abd3, then on the emulator with clean data (A.19):

```
$ grep -rn 'probe_enabled' app/src/main/java
SettingsSheet.kt:213: mutableStateOf(peerPrefs.getBoolean("probe_enabled", false))
SettingsSheet.kt:235: peerPrefs.edit().putBoolean("probe_enabled", on).apply()
```

```
VpnViewModel.kt:501: if (!prefs.getBoolean("probe_enabled", false)) return
uiautomator, Settings sheet on a fresh install:
'Help RostamVPN Improve' ... '' class=View checkable=true checked=false [880,1558][1017,1684]
$ adb shell run-as com.rostamvpn.android cat shared_prefs/rostam_prefs.xml | grep -c probe_enabled
0
```

Impact (before the fix): the UI asserted a consent the user had not given; once a signed configuration enabled the relay, users who had toggled the switch would relay gateway-chosen connections from their own address (see RVPN-01-317 for the relay design).

Preconditions: none for the display; activation additionally required `probe.enabled` in a signed configuration.

Recommendation: keep a single helper for the preference so that its default cannot diverge again, and add a unit test asserting that a fresh install reports `false`. The relay's description still does not say that third-party connections are relayed from the user's address; that wording issue is tracked in RVPN-01-317.

Classification: CWE-1188 · MASVS-PRIVACY-3 · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:P/VC:L/VI:N/VA:N/SC:N/SI:L/SA:N

RVPN-01-310 WP3: After the VPN is revoked the app keeps showing "VPN On" while traffic leaves directly (High)

Fix Note: This issue has been fixed by the development team in commit 264b167 and verified by Red Team Ltd to be working as expected. After a revocation from system Settings the app disconnected at once, showed "VPN Off", and the core's listener was gone (A.47); the watchdog now also requires a running tun (`VpnViewModel.kt:362-366`).

Note: User-visible change: a connect whose tun never comes up now ends in the Disconnected state (`VpnViewModel.kt:852-856`, `TunnelVpnService.kt:195-199`).

When Android revokes the VPN (the user disconnects it in system Settings, or another VPN app is activated with the user's consent), `onRevoke()` calls only `stopVpn()`, which closes the tun and clears `TunnelVpnService.isRunning` (`TunnelVpnService.kt:430-461, 536-539`). Nothing informs the view model: the connection state stays `Connected` and the core keeps serving SOCKS5. The watchdog cannot notice, because both of its liveness signals are independent of the tun. It treats 4 KB of received bytes on the app's own UID as proof of life (`VpnViewModel.kt:354-362`), and that counter covers the carrier sockets, not the tun (`VpnViewModel.kt:443-462`); its probe goes to the loopback SOCKS port (`VpnViewModel.kt:304-329, 364`), which keeps working without a tun. The same blind spot exists at connect time, where the result of `awaitTunReady()` is discarded (`VpnViewModel.kt:841`) and an `establish()` failure only logs (`TunnelVpnService.kt:197-200`); those two paths were reviewed statically.

Affected: `TunnelVpnService.kt:430-461` (`stopVpn`), `:536-539` (`onRevoke`); `VpnViewModel.kt:304-393` (`tunnelAlive`, `startHealthWatchdog`), `:437-462`, `:841`.

Proof of Concept: connected on the emulator, then revoked from system Settings (A.22):

```
19:26:17 Settings > Network & internet > VPN > RostamVPN > Disconnect
RostamVPN: Stopping VPN / VPN stopped (shutdown deferred) / VPN stopped (background)
$ adb shell dumpsys connectivity | grep -c 'type: VPN' -> 0
$ adb shell ip addr show tun0 -> Device "tun0" does not exist.
$ adb shell ip route get 192.0.2.1 uid 10140
192.0.2.1 via 10.0.2.2 dev wlan0 table 1012 src 10.0.2.16 uid 10140
RostamVPN UI: "VPN On", "Tap to disconnect"
/proc/net/tcp: LISTEN 127.0.0.1:1080 uid=10183 (core still serving)
19:28:38 (+141 s, more than two 45 s watchdog intervals):
UI "VPN On" | VPN networks: 0 | no [HEALTH] log lines
```

The route lookup for another app's UID shows that its traffic now uses the Wi-Fi interface directly.

Impact: the user reads "VPN On" while every app's traffic leaves the device unencrypted from the real address. On a censored network this exposes destinations and content to the network operator and can tie the user to sensitive services, with no indication on screen. This is silent tunnel failure as defined by the engagement standard.

Preconditions: a revocation event. It is not rated Critical because a revocation needs either a user action in system Settings or the user's consent to another VPN app; both are common among users who alternate between circumvention tools.

Recommendation: publish the service lifecycle to the view model and act on it; stop the core and move to `Disconnected` or `Failed` on revoke; make the watchdog require a live VPN network rather than loopback reachability:

```
override fun onRevoke() {
   stopVpn()
   TunnelEvents.state.value = TunnelEvents.State.REVOKED // observed by VpnViewModel
   super.onRevoke()
}
// VpnViewModel: on REVOKED -> RedSocketBridge.stop(); _connectionState.value = Disconnected
// watchdog: alive = TunnelVpnService.isRunning && vpnNetworkPresent() && tunnelAlive()
```

A `ConnectivityManager.NetworkCallback` filtered on `TRANSPORT_VPN` gives `vpnNetworkPresent()`. Treat `awaitTunReady ()== false` as a failed connect.

Classification: CWE-636 · MASVS-NETWORK-1, MASVS-PLATFORM-3, OWASP Mobile Top 10 2024 M8 · CVSS:4.0/AV:L/AC:L/AT:P/PR:N/UI:P/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-311 WP3: Exported activity acts on unauthenticated control intents and file URIs (Low)

Note: Partially fixed in commits f2c5b05, cfa276b and 3c82d4f and verified by Red Team Ltd: the `select_protocol` and `request_vpn` extras are ignored, `file://` URIs and the app's own provider are refused, and a code from a link is claimed only after the user confirms a dialog (A.48, A.49). Remaining: account codes still arrive through the unverifiable `rostam://` scheme.

`MainActivity` is exported for the launcher, a `rostam://c` deep link, `.rostam` files and `ACTION_SEND` (`AndroidManifest .xml:34-69`). Any app can therefore deliver three kinds of input that the activity acts on without confirmation. The `select_protocol` extra is persisted and forces a full reconnect (`MainActivity.kt:248-251`, `VpnViewModel. kt:1020-1045`). A `rostam://c?code=` link posts the code with the device's cid to `/api/v1/auth/claim` (`MainActivity .kt:258-264`, `VpnViewModel.kt:2471-2482`). And any `ACTION_VIEW` URI, including `file://` paths inside the app's own data directory, is opened with the app's privileges and fed to the configuration parser, with its first 40 characters written to logcat (`MainActivity.kt:276-280, 296-333`). The `rostam://` scheme is not verifiable, so any app may register it.

Affected: `MainActivity.kt:238-333` (`onNewIntent`, `handleDeepLink`, `handleRostamFileUri`); `AndroidManifest.xml:45-68 .`

Proof of Concept: all intents were sent from the adb shell, UID 2000 (A.23, A.25, A.27):

```
$ adb shell am start -n com.rostamvpn.android/.MainActivity --es select_protocol wgtcp
VpnViewModel: [SELECT] User selected protocol: wgtcp (was: auto)
VpnViewModel: [SWITCH] Protocol changed while connected, reconnecting...
19:29:41.222 Stopping VPN ... 19:29:41.241 VPN interface established, fd=79
rostam_prefs.xml: <string name="selected_protocol">wgtcp</string>
$ adb shell am start -n com.rostamvpn.android/.MainActivity -a android.intent.action.VIEW \
   -d 'rostam://c?code=RVPN01-TEST-CODE'
cashier stub: POST /api/v1/auth/claim {"code": "RVPN01-TEST-CODE", "cid": "00f70560..."}
$ adb shell am start -n com.rostamvpn.android/.MainActivity -a android.intent.action.VIEW \
   -d file:///data/data/com.rostamvpn.android/shared_prefs/redsocket.xml
RostamMain: Config file imported: <?xml version='1.0' encoding='utf-8' sta... (289 chars)
UI toast: "not a config code"
```

An implicit `rostam://c` intent opened the system chooser, because two other builds from the same code base on the AVD registered the scheme too. The earlier review stated that each forced reconnect leaves traffic unprotected for several seconds; the measured replacement of the tun took 19 ms between the two log lines and was below the 50 ms resolution of a poll on `/sys/class/net/tun0`, so that claim is not carried over.

Impact: a co-resident app can churn the tunnel at will and persist the protocol choice (which would move the user to the dormant WGTCP transport once a configuration enables it, RVPN-01-317); can bind a code of its

choosing to the device's cid (the effect depends on cashier semantics, not assessed); can make the app read its own private files, although no path back to the caller was found. Account codes distributed as `rostam://` links can be intercepted by any app that registers the scheme.

Preconditions: a co-resident app, or a web page the user follows (the deep link is `BROWSABLE`).

Recommendation: drop the `select_protocol` and `request_vpn` extras from external intents; ask the user before claiming a code; accept only `content://` URIs from `ACTION_SEND` or the document picker and reject `file://` and the app's own authority; deliver account codes as verified Android App Links on an HTTPS domain.

Classification: CWE-926 · MASVS-PLATFORM-1, OWASP Mobile Top 10 2024 M8 · CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC

RVPN-01-312 WP3: Always-on VPN starts the service but no tunnel comes up (Medium)

Note: Open at 5c0d00a: with always-on enabled the system again started the service with `android.net.VpnService` and no tunnel came up (A.50).

When the user enables Android's "Always-on VPN" for the app, the system starts `TunnelVpnService` with the action `android.net.VpnService`, and it does the same at every boot. `onStartCommand()` handles only the app's own `START_VPN` and `STOP_VPN` actions and returns without doing anything for any other action (`TunnelVpnService.kt:133–152`). The manifest does not opt out of always-on, so the system offers the setting.

Affected: `android/gen/android/app/src/main/java/com/rostamvpn/android/TunnelVpnService.kt:133–152`; `AndroidManifest.xml:81–93`.

Proof of Concept: on the emulator, with the app disconnected (A.24):

```
Settings > VPN > RostamVPN > Always-on VPN: on
$ adb shell settings get secure always_on_vpn_app -> com.rostamvpn.android
$ adb shell dumpsys activity services com.rostamvpn.android
  ServiceRecord{... com.rostamvpn.android/.TunnelVpnService}
  intent={act=android.net.VpnService pkg=com.rostamvpn.android} startRequested=true
$ adb shell dumpsys connectivity | grep -c 'type: VPN' -> 0
Block connections without VPN: on (confirmed "Turn on")
$ adb shell ip route get 192.0.2.1 uid 10140 -> RTNETLINK answers: Permission denied
$ adb shell 'nc -w 3 10.0.2.2 20312' -> nc: connect: Permission denied
RostamVPN UI: "VPN Off"; a manual Connect then works (route via tun0)
```

Afterwards lockdown was cleared with `settings put secure always_on_vpn_lockdown 0` and always-on was switched off on the same screen; both switches then read off, `always_on_vpn_app` returned `null`, and the AVD state was discarded at shutdown.

Impact: with always-on alone, a user who relies on the platform to reconnect after a reboot gets no VPN, while the system setting suggests protection, and traffic flows in the clear. With lockdown, which is the only kill switch available (the app has none of its own), the device stays offline until the user opens the app and connects by hand. The failure is closed in that case, but users who need the kill switch most cannot use it in a working configuration.

Preconditions: the user enables always-on for the app; reboot or process restart.

Recommendation: treat a start without the app's own action (`intent == null` or `intent.action == VpnService.SERVICE_INTERFACE`) as "connect with the saved settings" and drive the same connect sequence from a service-side controller, including the core start. Until that exists, declare `<meta-data android:name="android.net.VpnService.SUPPORTS_ALWAYS_ON" android:value="false"/>` in the service so that the system does not offer the setting.

Classification: CWE-754 · MASVS-NETWORK-1, OWASP Mobile Top 10 2024 M8 · CVSS:4.0/AV:L/AC:L/AT:P/PR:N/UI:P/VC:

RVPN-01-313 WP3: Debug reports are encrypted with a symmetric key shipped in the client (Low)

Note: Open at 5c0d00a: the report now goes to `cache/debug/` (commit 8709f28) and is uploaded through the tunnel when one is up, but it still decrypts with the zero key compiled into the client (A.54).

"Share Debug Log" first posts the report to a diagnostics host, directly from the device when the tunnel is down (`VpnViewModel.kt:1285–1317`), and otherwise writes it to the cache directory encrypted with AES-256-GCM under `DEBUG_REPORT_KEY` and opens the share sheet (`VpnViewModel.kt:1254–1282, 1217–1224`). The key is a constant in the APK; in this tree it is 32 zero bytes (`VpnViewModel.kt:50–52`), and a production key compiled in the same way is recoverable by anyone holding the APK. The report includes device model, OS and app version, network type, the connection diagnostics with the endpoint list and timezone, and up to 3,000 lines of the app's own logcat (`VpnViewModel.kt:1130–1142, 1176–1214`).

Affected: `VpnViewModel.kt:50–52, 1130–1142, 1176–1317`; `res/xml/file_paths.xml:3–5`.

Proof of Concept: export on the emulator, then decryption on the host with a single-file Java program using the in-tree key (A.28):

```
VpnViewModel: [debug] diag post failed: java.net.UnknownHostException: Unable to resolve
   host "diag.diag.example.net": No address associated with hostname
cache/rostam-debug-20261014-193547.enc (696 bytes)
$ java Decrypt.java debug-report.enc debug-report.txt
decrypted 668 bytes
Model: Google sdk_gphone_arm64 | Android: 11 (API 30) | App Version: 6.7.3 (466) | Network: WiFi
{"carrier":"","connections":0,"core":"http-only","endpoints":["10.0.2.2:20310"],...,
 "timezone":"America/Los_Angeles"}
--- Recent Logs (logcat) ---
```

The upload attempt failed only because the harness isolates DNS. The logcat section was empty on the API 30 image, so its content on production devices was not verified.

Impact: any report a user shares through a messenger or e-mail can be read by anyone with the APK, including the censor; it reveals device details, the endpoint list and timezone, and potentially the app's logs. The direct upload identifies the app on the network when the tunnel is down.

Preconditions: the user shares a report and the file reaches a third party.

Recommendation: encrypt to an operator public key (HPKE, RFC 9180, or an X25519 sealed box) whose private key never ships; send reports only through the tunnel; drop the endpoint list and redact identifiers from the log section.

Classification: CWE-321 · MASVS-CRYPTO-2, MASVS-STORAGE-2, OWASP Mobile Top 10 2024 M10 · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:A/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-316 WP3: Main activity is shown and usable over the lock screen (Medium)

Fix Note: This issue has been fixed by the development team in commit ca6f58e and verified by Red Team Ltd to be working as expected. With a PIN set and the app in front, waking the device showed the keyguard, focus on the notification shade and no app element (A.52).

`MainActivity.onCreate()` calls `setShowWhenLocked(true)` and `setTurnScreenOn(true)` unconditionally on API 27 and later (`MainActivity.kt:204–207`). If the app is in the foreground when the device locks, or is brought up from its notification, the full interface is displayed above the keyguard and accepts input.

Affected: `android/gen/android/app/src/main/java/com/rostamvpn/android/MainActivity.kt:204–207`.

Proof of Concept: on the emulator with a PIN set (A.30):

```
$ adb shell locksettings set-pin 1234
$ adb shell am start -n com.rostamvpn.android/.MainActivity
$ adb shell input keyevent KEYCODE_SLEEP ; adb shell input keyevent KEYCODE_WAKEUP
$ adb shell dumpsys window | grep mCurrentFocus
  mCurrentFocus=Window{8268244 u0 com.rostamvpn.android/com.rostamvpn.android.MainActivity}
$ adb shell dumpsys activity activities | grep mKeyguardShowing
    mKeyguardShowing=true
uiautomator: "VPN Off", "Tap to connect", content-desc "Settings"
tap Settings while locked -> "Protected Apps", "Help RostamVPN Improve", "Share Debug Log"
$ adb shell locksettings clear --old 1234
```

Impact: a person holding a locked phone, for example at a checkpoint, sees that a circumvention VPN is installed and in use, can connect, disconnect or change the region, change the relay consent and start the debug export, all without the PIN.

Preconditions: physical access to a locked device on which the app was in the foreground or is opened from its notification.

Recommendation: remove both calls. If a lock-screen surface is wanted, use a notification with no actions on the secure lock screen (`Notification.VISIBILITY_SECRET` or a public version without the product name).

Classification: CWE-284 · MASVS-PLATFORM-3, OWASP Mobile Top 10 2024 M8 · CVSS:4.0/AV:P/AC:L/AT:P/PR:N/UI:N/VC

RVPN-01-319 WP3: Replacing the tun during a reconnect can leave the VPN network without routes (Medium)

This item was found during the version 1.1 re-verification at 5c0d00a. Every reconnect that happens while the service is running (a watchdog reconnect, a split-tunnel change, or Connect pressed from the failed state) sends `ACTION_STOP` and then `ACTION_START` back to back (`VpnViewModel.kt:816–843`). `stopVpn()` hands the old interface to a background thread, which stops the bridge and closes the descriptor only if no newer generation has started (`TunnelVpnService.kt:406–436`). Meanwhile `startVpn()` increments the generation and establishes the replacement on the main thread (`TunnelVpnService.kt:154–199`). The two run concurrently. In one observed case the old interface disappeared while the replacement was being created. The replacement reused the name `tun0`, and Android 11 kept the VPN's policy rules pointing at the old interface's routing table, which the kernel had emptied. When the generation check wins instead, the superseded descriptor is never closed and interfaces accumulate (`tun0` and `tun1` side by side were observed). The pattern predates the remediation; the remediated flow only made its outcome visible.

Affected: `TunnelVpnService.kt:154–199` (`startVpn`), `:406–436` (`stopVpn`); `VpnViewModel.kt:816–843` (`connectAuto`), `:1067–1086` (`disconnectThenReconnect`).

Proof of Concept: the exit was stopped while connected, and the watchdog's reconnect then failed (A.55):

```
00:26:23.182 [HEALTH] tunnel is dead, auto-reconnect 1/3
00:26:23.252 RostamVPN: Stopping VPN
00:26:23.270 TProxyStopService called (background thread)
00:26:23.279 VPN interface established, fd=63 (main thread)
00:26:23.291 VPN stopped (background) (old descriptor closed)
UI: "Connection Failed"; dumpsys connectivity: VPN CONNECTED,
   Routes [ 0.0.0.0/0 -> tun0, ::/0 -> tun0 ]
ip rule: ... uidrange 0-10182 lookup 1016 ip route show table 1016: (empty)
new tun0: kernel routes only, in table 1018
ip route get 192.0.2.1 uid 10140 -> via 10.0.2.2 dev wlan0 table 1012
ip -6 route get 2001:db8::1 uid 10140 -> via fe80::2 dev wlan0 table 1012
```

Fifteen further reconnects routed correctly: eight split-tunnel changes, five fresh connects each followed by one reconnect, one connect from the failed state, and a second watchdog failure (A.57). The broken state was therefore seen once in sixteen reconnects. The mechanism above is inferred from the logs and the routing state, not from platform source.

Impact: while the state lasts, every routed app's traffic leaves through the underlying network from the real address although Android shows a connected VPN. That defeats the fail-closed behaviour the app relies on after a failed reconnect (A.32). The observed case ended at "Connection Failed"; if the race hit a reconnect that then succeeds, the app would show "VPN On", which was not observed. A censor that blocks an exit triggers exactly this reconnect path.

Preconditions: a reconnect while the service is running, and the race resolving as above; observed on the Android 11 emulator only.

Recommendation: keep the interface across reconnects and restart only the core and the bridge; where the interface must change, establish the replacement first and close the old descriptor on the same thread afterwards; never leave a superseded descriptor open; after `establish()`, confirm through `ConnectivityManager` that the VPN network carries its default routes, and re-establish otherwise.

```
val old = vpnInterface
val fresh = establishTunnel(allowedApps, appListMode, plan) ?: return failClosed()
vpnInterface = fresh
runCatching { hev.htproxy.TProxyService.getInstance().TProxyStopService() }
old?.close() // same thread, after the replacement exists
```

Classification: CWE-362 · MASVS-NETWORK-1, OWASP Mobile Top 10 2024 M8 · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC

Miscellaneous Issues

This section lists weaknesses without a demonstrated exploit path, hardening gaps, non-conformance and dependency hygiene.

RVPN-01-301 WP3: Deployment identifiers remain in the public tree (Low)

Fix Note: This issue has been fixed by the development team in commit b69874a and verified by Red Team Ltd to be working as expected. A side effect of b69874a on a vendored smoltcp file broke the offline WGTCP build; commit 621ba38 restored the file, and both cores built again at 5c0d00a (A.37).

Commit f60abd3 replaced most deployment identifiers with placeholders, and Red Team Ltd verified that part: the exit addresses in both configurations are RFC 5737 and RFC 3849 documentation addresses, the live-test defaults in `edge.rs` and `mux.rs` use documentation addresses, the contact mailbox is a role address on the brand domain (`brand_strings.xml:7`, `brands/rostam.json:91`), and the operational remark about shared origin addresses flagged by the earlier review is gone from `default-config.notes.md`, which is in any case excluded from the APK (`app/build.gradle.kts:42-44`). At f60abd3 the CDN host names listed below were still present; commit b69874a replaced them. Values are not reproduced here.

A sweep of all non-vendored text files for public IPv4 addresses outside the documentation ranges, CDN host names, OAuth client identifiers, mailboxes and 64-hex strings (A.14) produced the following, by class. Public addresses that did appear are test placeholders, public resolvers, or provider prefixes in routing unit tests, not operator infrastructure.

Location	Class of value	Status
`brands/configs/rostam.json:141-143`	three CloudFront distribution host names	replaced with `dexample` placeholders in b69874a
`brands/configs/rostam.json:320-322`	the same three host names	replaced in b69874a
`default-config.json:9`, `brands/configs/rostam.json:9`	host name under a commercial CDN's shared TLS domain	replaced with an `example` host name in b69874a
`default-config.json:29`, `brands/configs/rostam.json:29`, `redsocket-http-core/src/mux.rs:508` ↪ , 14 fixtures in `src/config.rs`	64-hex Noise public key	not a deployment identifier: per the development team, the public half of a published example key pair; retained
`redsocket-http-core/src/cfgseal.rs` ↪ `:18-23, 166-167`	Ed25519 verifying key and its sealed test fixture	public verification key and fixture, retained by design

The development team stated that the private half of the example key pair is published in the servers' example configuration. That file lies outside the WP3 repository and was not examined; the comment at `mux.rs:489`, which describes the test using this key as a session "against a production POP", predates that clarification. The verifying key is public by construction and is needed in every client.

Affected: `brands/configs/rostam.json:9, 141-143, 320-322`; `android/gen/android/app/src/main/assets/default-` ↪ `config.json:9` (at f60abd3).

Proof of Concept: at f60abd3, `python3 -I ident_sweep.py <repo>` reported 6 CloudFront and 2 Fastly hits outside placeholders, and a `diff` of the bundled and brand configurations showed the three distributions only in

the brand file. At b69874a the changed blobs match the working tree, the same lines carry placeholders, and the sweep reports no CDN host name outside placeholders and test values (A.34).

Impact (before the fix): a ready-made block list of the fleet's CDN fronts in a public repository.

Preconditions: none.

Recommendation: inject production front names at build time so that they cannot re-enter the tree, and treat the history before b69874a as published when deciding whether to rotate those fronts.

Classification: CWE-540 · OWASP Mobile Top 10 2024 M8 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N

RVPN-01-302 WP3: JNI boundary carries unused exports and unguarded entry points (Low)

Note: Partially fixed in commit 24aa10f, verified in code: the Kotlin side now catches `Throwable` around the tun bridge's native calls (`TunnelVpnService.kt:174, 401, 418, 502` at 5c0d00a). Remaining: the five unused JNI exports and the 25 exported functions without `catch_unwind` were unchanged in the rebuilt libraries (A.38).

The JNI surface binds correctly: the 29 `external fun` declarations of `RedSocketBridge.kt` and the 4 of `WgtcpBridge.kt` all resolve to exports of the built libraries. Three hygiene gaps remain. First, `libredsocket_core.so` exports four JNI functions that no Kotlin code declares (`clearBlockHistory`, `getBlockStats`, `setBlocklist`, `setUserRule`, `lib.rs:910–1000`), and `libwgtcp_core.so` exports `nativeGetConnectionStatus` without a declaration. Every exported name is a static string a censor can match in a decompiled library, and these add product vocabulary for no function. Second, 25 of the 34 exported functions in `lib.rs` have no `catch_unwind`, among them `rsCarriers` and `runDiagnostics`, which re-run `config::load` on the OTA document (`lib.rs:696, 725`), and `setClientId`, whose callee unwraps a mutex (`lib.rs:94, 119`). A panic that reaches an `extern "system"` boundary aborts the process, taking the VPN down; the WGTCP shell and the Apple FFI are fully guarded. No network-reachable panic was found in these paths. Third, the Kotlin side catches `Exception` around the bridge's native calls (`TunnelVpnService.kt:173–178, 422–427, 439–444`), so an `UnsatisfiedLinkError`, which is an `Error`, would escape `onStartCommand`. That last point is static only: the APK without the bridge library was built but not run.

Affected: `redsocket-http-core/src/lib.rs:327–1000`; `wgtcp-android/src/lib.rs`; `TunnelVpnService.kt:173–178, 422–427, 439–444`.

Proof of Concept: A.5:

```
$ llvm-nm --defined-only -D libredsocket_core.so | grep -c ' Java_' -> 33
$ diff kotlin-externals.txt so-exports.txt
> clearBlockHistory
> getBlockStats
> setBlocklist
> setUserRule
$ llvm-nm --defined-only -D libwgtcp_core.so: 5 Java_ exports, 4 declared (+ nativeGetConnectionStatus)
script over lib.rs: 34 exported JNI functions; 25 without catch_unwind
```

Impact: a larger static signature, and process termination if any of the unguarded paths ever panics.

Preconditions: a panic in an unguarded path; for the signature, possession of the APK.

Recommendation: remove or feature-gate the unused exports; wrap every exported body as `start` does and use `lock().unwrap_or_else(|e| e.into_inner())`; catch `Throwable` around the bridge in Kotlin and fail the connect cleanly.

Classification: CWE-248 · MASVS-CODE-4, MASVS-RESILIENCE-3 · CVSS:4.0/AV:L/AC:H/AT:P/PR:N/UI:N/VC:N/VI:N/VA:L/

RVPN-01-303 WP3: Native libraries embed the builder's user name, and the documented build bypasses the vendoring (Low)

Note: Partially fixed in commit 97d25fc, verified: `README.md` now builds each core from its crate directory, where the vendored sources and the remapping apply. Remaining: the remapping still covers only the home-directory prefix; the rebuilt libraries again carried 32 and 20 `/build/<user>/` strings (A.38).

All three `.cargo/config.toml` files remap only the top-level prefix of macOS home directories to `/build` (`redsocket-http-core/.cargo/config.toml:4, 9, 14`, `wgtcp-android/.cargo/config.toml:2`, `vendor/redsocket_wgtcp/.cargo/config.toml:2`). The user name and everything below it survive in panic-location strings, so a library built on the maintainers' machines carries `/build/<user>/` followed by the toolchain and checkout paths, which reveal the build host's OS and architecture and are identical across brands built from the same checkout. Separately, the documented build command (`README.md:59-62`) runs cargo-ndk from the repository root with `--manifest-path`. With cargo-ndk 4.1.2 that command fails, and with Cargo itself a root-level invocation does not read the crate's `.cargo/config.toml` at all, so neither the vendored sources nor the remapping nor `--cfg chacha20_force_neon` apply.

Affected: the three `.cargo/config.toml` files; `README.md:59-62`.

Proof of Concept: A.4 and A.6:

```
$ strings -a libredsocket_core.so | grep -c '^/build/' -> 32 (libwgtcp_core.so: 20)
/build/<user>/.rustup/toolchains/stable-aarch64-apple-darwin/lib/rustlib/src/rust/library/...
$ cargo ndk -t arm64-v8a -o $JNI --manifest-path redsocket-http-core/Cargo.toml build --release
error: `cargo metadata` exited with an error: error: could not find `Cargo.toml` in
     `/private/tmp/rvpn01/wp3/tree` or any parent directory
$ cargo metadata --manifest-path redsocket-http-core/Cargo.toml --offline (from the root)
{'registry': 148, ...} (from the crate directory: {'vendor': 148, ...})
```

Impact: a stable, product-linking static string in every shipped library, and builds that silently differ from the reviewed configuration depending on the working directory.

Preconditions: possession of the APK; a build run as documented.

Recommendation: remap the full home directory and the checkout (for example `--remap-path-prefix=$HOME=/home/build` and `--remap-path-prefix=$PWD=/src`, set by a build script rather than a static file); correct the README to `cd redsocket-http-core && cargo ndk ...`; check the shipped libraries for absolute paths in CI.

Classification: CWE-497 · NIST SP 800-218 PW.6, MASVS-RESILIENCE-3 · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/

RVPN-01-309 WP3: Stable device identifiers reach every exit and the cashier (Medium)

Note: Partially fixed in commit fa567f1 and verified by Red Team Ltd: failure logs no longer carry the cid (A.53). Commit 34a5a11 had removed the advertising ID, the `AD_ID` permission and the advertising-ID relay tag (A.39, A.45); the development team restored all three in 49ac756, stating that it uses the identifier to detect fraud and click farms (A.58). The cashier therefore receives the advertising ID with the cid at registration again, and the relay tag is the advertising ID. Remaining: the SSAID-derived cid and the timezone still reach every exit (A.45), the cid remains the Play account identifier, and the policy and payment wording are unchanged. Red Team Ltd notes that the Play Console advertising-ID declaration and the Data Safety form must name fraud prevention as a purpose, and that the Play policy allows the advertising ID to be associated with a persistent device identifier such as the SSAID-derived cid only with the user's explicit consent.

The cid is `HMAC-SHA256("rostam-cid-v1", ANDROID_ID | applicationId)` truncated to 8 bytes (`CashierIdentity.kt:26, 59-85`). ANDROID_ID is stable per signing key and device, so the cid survives reinstallation and cannot be reset by the user. It travels with the timezone in the first encrypted frame of every carrier (`tunnel.rs:105-110`), so every exit sees it. The cashier receives it in registration, in usage reports and in a URL query string, in code claims and in web orders (`CashierApi.kt:69-98, 119-123, 186-196`), and it is set as the Play `obfuscatedAccountId` (`VpnViewModel.kt:2256`, `PlayBilling.kt:146`). The app requests the advertising ID (`AndroidManifest.xml:5`, `Gaid.kt:20-42`) although it shows no advertising (`README.md:34`), sends it to the cashier at registration, and uses it as the relay tag. Endpoint order, decoy `Host` and path are derived from the cid (`config.rs:24, 34, 630`), so a device presents the same first endpoint and `Host` on every session. The privacy policy mentions "unique device identifiers" generically (`PrivacyPolicyScreen.kt:127-132`), and the crypto paywall says "nothing links the payment to you" (`res/values/strings.xml:159`) while the order carries the cid.

Affected: as listed.

Proof of Concept: the test exit decrypted the hello of every carrier from the emulator (A.20), and the cashier stub recorded the registration (A.18):

```
hello frame 59 bytes ciphertext; plaintext: META|America/Los_Angeles|||00f70560...
hello frame 60 bytes ciphertext; plaintext: MUX2|America/Los_Angeles|||00f70560...|
POST /api/v1/cid/register {"cid": "00f70560...", "brand": "rostam", "tz": "America/Los_Angeles",
    "ifa": "320d31f1-...", "lmt": 0}
UPGRADE ... Host: teams.microsoft.com (every carrier from this install)
```

The Auto Backup archive (RVPN-01-315) also held six queued Play Billing Library telemetry events for Google's Clearcut backend with model, build fingerprint, country, locale and MCC/MNC metadata; their transmission was not observed because DNS was isolated.

Impact: a compromised or seized exit can link all sessions of a device over months and across reinstalls; the cashier's records join cid, advertising ID and payments, so seizure of either side links identity to usage.

Preconditions: access to an exit or to cashier records.

Recommendation: use a random per-install cid with a user-visible reset; give exits a short-lived session token instead of the cid; drop the timezone from the hello or coarsen it; stop collecting the advertising ID and remove the `AD_ID` permission; correct the payment wording and name the identifiers in the policy.

Classification: CWE-359 · MASVS-PRIVACY-1, MASVS-PRIVACY-2, MASVS-PRIVACY-3, OWASP Mobile Top 10 2024 M6 · CVSS:4.0/AV:N/AC:L/AT:P/PR:H/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

RVPN-01-314 WP3: Signed configuration has no freshness, slot or brand binding (Medium)

Note: Partially fixed in commit 14f3125, verified in code: the direct download is bounded at 256 KB (`RemoteConfig.kt:225`). Remaining: no sequence, expiry, brand or slot in the signed body, the permanent `config_no_v2` flag, and the stop at the first answering base.

The configuration path gets integrity right: `cfgseal::unseal()` refuses anything whose Ed25519 signature over the JSON does not verify, and a refused document keeps the previous one (`cfgseal.rs:102-113`, `RemoteConfig.kt:256-264`). What the signature covers is the problem. It covers the JSON body only, and nothing in the client requires the body to be newer than the stored one, unexpired, or issued for this brand or for the v2 rather than the v1 slot. `storeSealed()` accepts any verifying document that has a `version` and at least one endpoint (`RemoteConfig.kt:247-277, 352-359`); the version and date shown to the user come from the unauthenticated `X-Config-Version` and `X-Config-Published` headers (`RemoteConfig.kt:215-217, 226-227`). A single 404 on `/api/v2/config` sets `config_no_v2` permanently (`RemoteConfig.kt:181-186`). The refresh loop stops at the first base that answers anything other than a network error, so a 200 carrying an invalid body suppresses every fallback and mirror (`RemoteConfig.kt:71-75`). The direct fetch reads the body without a bound (`RemoteConfig.kt:225`); the mirror path is bounded at 256 KB. There is no certificate pinning and no network security configuration, so on API 23 devices user-installed CAs are trusted. The encryption layer offers no confidentiality, since its key is derived from constants in the client (`cfgseal.rs:25-67`), and the module comment "verify, then decrypt" (`cfgseal.rs:4`) does not match the code, which decrypts first.

Affected: `redsocket-http-core/src/cfgseal.rs:4, 25-114`; `RemoteConfig.kt:69-89, 179-277, 352-359`.

Proof of Concept: a scratch crate opened the publisher-sealed vector that ships in the core's tests (A.13), and the downgrade flag was observed on the emulator (A.17, A.26):

```
$ cargo run --offline -- <signing-cert hash from cfgseal.rs:167>
verified OK today; 91 bytes JSON; top-level keys: ["transport", "version"]
  field expires / expires_at / not_after / seq / serial / published_at / issued_at
      / brand / slot present: false
tampered copy refused: true
first launch: GET /api/v2/config -> 404 ; GET /api/v1/config -> 404
redsocket.xml: name="config_no_v2" value="true"
cold relaunch: GET /api/v1/config only
```

Replay itself was not exercised: the testing team holds no signing key and did not intercept TLS.

Impact: an adversary that can answer for any configuration base, fallback or image mirror (a compromised front, a seized fallback domain, or TLS interception where the device trusts the adversary's CA) can replay any

configuration ever signed, for example one listing retired endpoints it now controls, an older `api` block, or a relay policy, and can keep it in place indefinitely, pin the client to the v1 slot, or suppress updates.

Preconditions: control of a response on the configuration path.

Recommendation: add `seq`, `expires_at`, `brand` and `slot` to the signed JSON and refuse any document whose `seq` is not greater than the stored one, which is expired, or which is for another brand or slot; display only signed values; continue to the next base on a verification failure; let `config_no_v2` expire; bound the direct read with the existing `readBoundedBytes`.

Classification: CWE-294 · MASVS-NETWORK-1, MASVS-NETWORK-2, MASVS-CODE-4 · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/

RVPN-01-315 WP3: Auto Backup carries identity, policy state and the tunnel log (Low)

Fix Note: This issue has been fixed by the development team in commits dcf687c, 46ee131 and 8709f28 and verified by Red Team Ltd to be working as expected. The local backup transport declined the package as empty, the development team's `adb backup` archive held only the manifest, release builds configure the bridge at `warn` without a log file (`TunnelVpnService.kt:391–395, 523–526`, unit test `HevConfigTest`), and the FileProvider covers only `cache/debug/` (A.42, A.51).

The application allows backup (`AndroidManifest.xml:27–30`) and both rule files exclude only `wgtcp_identity.xml` ↪ (`backup_rules.xml:4`, `data_extraction_rules.xml:5, 8`). Cloud backup and device-to-device transfer therefore carry the cid and its origin, the last working endpoint, payment and usage state, the protocol choice, the bridge configuration and the bridge log, which the app writes at `debug` level in every build (`TunnelVpnService.kt:391, ↪ 400–401`). Two further files are written to the backed-up area by code and were absent in the test only because the harness produced no data for them: `files/remote-config.json`, the decrypted configuration (`RemoteConfig. ↪ kt:265`), and `rostam_settings`, the split-tunnel app list and bypass prefixes (`TunnelVpnService.kt:48–51`). A restored cid is reused as is on the new device (`CashierIdentity.kt:40–42`). The FileProvider exposes the whole files, cache and external storage roots (`file_paths.xml:3–5`), although only one cache file is ever shared; it is not exported.

Affected: `AndroidManifest.xml:27–30`; `res/xml/backup_rules.xml:4`; `res/xml/data_extraction_rules.xml:5, 8`; `TunnelVpnService.kt:391–401`; `RemoteConfig.kt:265`; `res/xml/file_paths.xml:3–5`.

Proof of Concept: A.29:

```
$ adb shell bmgr transport com.android.localtransport/.LocalTransport
$ adb shell bmgr backupnow com.rostamvpn.android -> Backup finished with result: Success
$ tar -tvf backup-full.tar
apps/com.rostamvpn.android/f/hev-socks5-tunnel.yml
apps/com.rostamvpn.android/f/hev-socks5-tunnel.log
apps/com.rostamvpn.android/db/com.google.android.datatransport.events
apps/com.rostamvpn.android/sp/rostam_prefs.xml
apps/com.rostamvpn.android/sp/redsocket.xml
redsocket.xml keys: cid_from_ssaid redsocket_cid redsocket_last_ok config_no_v2
rostam_prefs.xml keys: paid data_total selected_protocol entry_done review_* ...
datatransport: 6 events, transport PLAY_BILLING_LIBRARY, backend cct
```

The bridge log was empty because the stub writes none; what hev-socks5-tunnel records at `debug` could not be verified without its binary.

Impact: an adversary with access to the user's cloud backup, or to a device the data was transferred to, learns that RostamVPN was used and obtains the cid that links the user to exit and cashier records; with a cached configuration and split-tunnel list it also obtains the endpoint list and the user's app choices.

Preconditions: access to the backup account or a restored device.

Recommendation: exclude `sharedpref`, `file` and `database` domains from both `cloud-backup` and `device-transfer`, or set `allowBackup="false"`; run the bridge at `warn` with no log file in release builds; narrow `file_paths.xml` to a single `debug/` subdirectory of the cache.

Classification: CWE-530 · MASVS-STORAGE-2, OWASP Mobile Top 10 2024 M9 · CVSS:4.0/AV:N/AC:H/AT:P/PR:H/UI:N/VC:

RVPN-01-317 WP3: Dormant features ship unauthenticated trust paths (Medium)

Note: Partially fixed in commits 5c0d00a, 82623d9 and 34a5a11 verified in code and by the vendored crate's tests (A.41, A.56): the WGTCP carrier now reports up only after the first WireGuard handshake (`vendor/redsocket_wgtcp/src/carrier.rs:210–212`), its exported status reports `reconnecting` until a handshake completes (`wgtcp-android/src/lib.rs:56–60`, an export this app does not call), the sender index is random (`wg.rs:79–87`), carrier writes are bounded (`carrier.rs:43, 368`), SOCKS5 streams are capped (`socks5.rs:32, 58, 103`), the configuration is validated (`config.rs:121–125, 193`), and the relay tag was a random value between 34a5a11 and 49ac756; since 49ac756 it is the advertising ID again (A.58). Remaining: WGTCP still resolves names locally (`socks5.rs:94, 211–215`), its profile is unsigned and its record header deterministic; control frames and the blocklist are unsigned; the relay gateway is unauthenticated and the consent text unchanged.

Three features are present in the shipped libraries, disabled by the bundled configuration, and can be enabled by a signed configuration without an app release. Each trusts an input that is not authenticated end to end. All three were reviewed statically; none was enabled during testing.

Control stream and blocklist (`protection` or `messaging` enabled). Frames the exit sends on a server-opened MUX2 stream are queued verbatim (`ctl.rs:385–437`, in particular `422–428`) and applied by Kotlin without a signature: `rules` replaces and persists the user's allow and deny lists, `entitlement` sets the paid state, `notice` posts a system notification with exit-chosen text, `cursor` is persisted (`VpnViewModel.kt:1923–1967`). The blocklist is fetched over plain HTTP through the exit (`blocklist.rs:801–805`) and a `/0` prefix is accepted (`blocklist.rs:201–216`).

Peer relay (`probe.enabled` and user consent). The device sends a plaintext `RMUX` hello carrying the app tag, the advertising ID or cid, and the timezone to a gateway reached through the current exit, and authenticates the gateway in no way (`peer.rs:390–421`). It then dials gateway-chosen targets from its own address, resolving names with the system resolver or a hard-coded public resolver (`peer.rs:548, 585–616, 700–720`). Port allow-lists, byte caps and hourly budgets exist. The consent text, "Share anonymous connectivity probes to help us find working transports faster" (`res/values/strings.xml:51`), does not say that third-party connections are relayed from the user's address.

WGTCP (`wgtcp` in `transport_order`). The vendored client resolves destination names with the system resolver in the app's UID, which the VPN excludes, so lookups leave in the clear (`vendor/redsocket_wgtcp/src/socks5.rs:121, 166–170`). The WireGuard profile, including the server key and endpoints, is fetched unsigned over platform TLS (`WgtcpProvisioner.kt:67–130`). The record header is a deterministic function of the length (`record.rs:37–56`), and the carrier is reported up when TCP connects, before any WireGuard handshake (`carrier.rs:100`).

Affected: as listed.

Proof of Concept: static only; code excerpts as cited. Exercising them would need a configuration signed by the operator.

Impact: with the control stream on, one compromised exit can rewrite persistent policy and entitlement on a device and push a phishing notification from the trusted app. With the relay on, a compromised exit or gateway obtains residential egress from consenting users' addresses tagged with the advertising ID. With WGTCP on, every destination name is visible to the network's resolver while the UI shows "Connected", which is deanonymisation.

Preconditions: a signed configuration enabling the feature, plus a hostile exit or gateway (control stream, relay) or a passive network observer (WGTCP).

Recommendation: sign control frames and the blocklist with the configuration key and reject `/0`; authenticate the gateway with Noise NK against a key in the signed configuration and drop the advertising ID; resolve WGTCP destinations inside the tunnel before enabling the transport, deliver its profile inside the signed configuration, and report the carrier up only after the first completed handshake.

Classification: CWE-345 · MASVS-NETWORK-1, MASVS-PRIVACY-3 · CVSS:4.0/AV:N/AC:H/AT:P/PR:H/UI:N/VC:H/VI:H/VA:

RVPN-01-318 WP3: Supply-chain controls for Gradle, the tun bridge and Rust dependencies (Low)

Note: Partially fixed in commits 4906b79, 6f89908 and 5c0d00a, verified: the wrapper pins `distributionSha256Sum` (the hash of the distribution used in A.2, not compared with Gradle's published value), release ProGuard

files are listed explicitly, `rand` and the yanked `chacha20` 0.10.1 are gone, `tun` is optional and the vendored library carries its licence (A.41). Remaining: no Gradle dependency verification, no mirror checksums, remote repositories still configured, the tun bridge still unpinned and outside the tree, the server module still compiled into the client library, and no SBOM.

The offline build works: from an empty Gradle user home with only the cached distribution, `--offline` completed against `local-maven` alone (A.2), and no dependency uses a dynamic version. The controls around that mirror are missing. There is no `gradle/verification-metadata.xml`, the mirror's 475 modules carry no checksum or signature files, and `google()`, `mavenCentral()` and `gradlePluginPortal()` remain configured after it (`settings.gradle:4-6`, `build.gradle.kts:7-8, 22-23`), so an online build silently fetches whatever the mirror lacks without verification. The wrapper has no `distributionSha256Sum` (`gradle/wrapper/gradle-wrapper.properties`). Release keep rules are collected with `fileTree("."){ include("**/*.pro")}` (`app/build.gradle.kts:104-108`), so any `.pro` file under the module, including build outputs, alters R8. `com.google.zxing:core` is declared but not referenced (`app/build.gradle.kts:239`).

The tun bridge, the component that sees every packet, is outside the tree: `README.md:8` says it is "built separately from its upstream source", with no version, commit or digest. The in-app notice lists its licence as GPL-3.0 (`OpenSourceScreen.kt:87-88`) while the project is Apache-2.0; no licence text or source offer for it was found in the tree. Licence compliance was not assessed legally.

On the Rust side, `cargo audit` reported no vulnerability; `chacha20` 0.10.1 is yanked in both WGTCP lock files, pulled in by `rand` 0.10.2, which the vendored crate uses only in its CLI binary (`vendor/redsocket_wgtcp/src/main.rs:60`). The library compiles its `server` module into the client (`vendor/redsocket_wgtcp/src/lib.rs:10`). No SBOM or build provenance is produced.

Affected: as listed.

Proof of Concept: A.2, A.9, A.10 and file checks:

```
verification-metadata.xml present: no | distributionSha256Sum present: 0
local-maven: 475 .pom, 194 .jar, 89 .aar, 0 checksum files, 0 .asc files
cargo audit (1,329 advisories): wgtcp-android 196 deps -> chacha20 0.10.1 yanked
cargo tree -i chacha20@0.10.1: rand v0.10.2 <- redsocket-wgtcp v0.1.0 <- wgtcp-android v1.0.0
```

Impact: an unverifiable native input and a mirror whose integrity rests on repository history alone; a release that cannot be attested or reproduced.

Preconditions: control of the build environment, a remote repository, or the separately built bridge library.

Recommendation: enable Gradle dependency verification with SHA-256 and PGP; add checksums to the mirror and remove remote repositories for release builds; add `distributionSha256Sum`; list ProGuard files explicitly; vendor hev-socks5-tunnel at a pinned commit with a build script, licence text and source offer; update `rand`; feature-gate the server module; publish a CycloneDX 1.7 or SPDX 3.0.1 SBOM and SLSA provenance, which also supports the Cyber Resilience Act's component documentation duties.

Classification: CWE-494 · NIST SP 800-218 PS.3, PW.4, OWASP Mobile Top 10 2024 M2 · CVSS:4.0/AV:N/AC:H/AT:P/PR:H/U

Conclusions

The RostamVPN Android application rests on a sound cryptographic core and a deliberately small platform surface, and most of its network-facing code withstood both review and hostile input. The weaknesses Red Team Ltd found are concentrated in two places: the glue between the Android VPN lifecycle and the transport core, and the transport's appearance on the wire. Four items reach High severity, and all four are reachable in the shipped configuration without any server-side change. The project is in a good position to fix them, because each has a narrow, local cause. The paragraphs below walk through every area in scope: what was examined, what held, what did not, and what to do first.

Remediation status (version 1.1). The development team answered the version 1.0 report within a day, with twenty commits on top of b69874a up to 5c0d00a and a follow-up, 82623d9. Red Team Ltd re-verified each in code and, where the behaviour can be observed, on the emulator (A.35 to A.57). Two of the four High items are fixed. Another app can no longer capture the tunnel through port 1080 (RVPN-01-305), and a revocation

now ends in “VPN Off” with the core stopped (RVPN-01-310). The other two are partially fixed. The carrier’s 2-byte segments are gone, and the remaining HTTP signature is to be retired by the Proteus transport (RVPN-01-304). Registration waits for the tunnel, while the configuration fetch at launch stays direct by design; the advertising ID, removed in 34a5a11, was restored in 49ac756 for fraud and click-farm detection (RVPN-01-307). Backups, the lock screen, the code-claim flow and the external intents are fixed or largely fixed. Three items saw no change: the unauthenticated SOCKS5 port (RVPN-01-306), the always-on start (RVPN-01-312) and the debug-report key (RVPN-01-313). The re-verification also exposed a race in how the tun is replaced during a reconnect, which once left the VPN network without routes (RVPN-01-319). The paragraphs below describe what was found at f60abd3, with the current state noted where it differs.

Build and reproducibility. The debug APK built offline from an empty Gradle home in about a minute, using only the in-tree mirror and a cached Gradle distribution. That is a stronger reproducibility property than most Android projects offer. Both Rust cores built in under twenty seconds each with their release profiles. The weaker points are around the mirror: it carries no checksums or signatures, Gradle dependency verification is off, the wrapper is unpinned, and remote repositories remain configured, so an online build may quietly diverge from the reviewed set (RVPN-01-318). The documented native build command does not work from the repository root with the current cargo-ndk, and a root-level Cargo invocation ignores the vendoring and path remapping entirely. The remapping that does apply leaves the builder’s user name and the toolchain path in every shipped library (RVPN-01-303). None of this affects users directly. It affects the project’s ability to attest what it ships, which matters for a tool whose distribution is itself a target.

Manifest, exported components and permissions. The permission set is lean: network access, foreground service, network state, billing, and the advertising ID permission. There is no location, contacts, storage or phone permission. Only `MainActivity` is exported. The VPN service is protected by `BIND_VPN_SERVICE`, the FileProvider is not exported, and the debug-only broadcast receiver is confined to the debug source set. Where the exported activity is concerned, the problem is not exposure but trust. The activity persists a protocol choice and forces reconnects on an external extra, claims an account code against the device’s identity without asking, and opens arbitrary `file://` URIs with its own privileges (RVPN-01-311). The measured cost of a forced reconnect was small (19 ms between closing and re-establishing the interface), which corrects the earlier review’s estimate of several seconds; the remaining risk is integrity of user choices, plus interception of account codes through an unverifiable custom scheme. The activity is also shown above the lock screen, which at a checkpoint discloses and operates the app without the PIN (RVPN-01-316).

Storage, backup and logging. The WireGuard identity is the only file excluded from backups, and it is the right file to exclude. At f60abd3 everything else backed up: the cid and its origin, payment and usage state, the protocol choice, the bridge configuration and log, and Google’s queued billing telemetry (RVPN-01-315); the decrypted configuration and the split-tunnel list would join them in normal use. The Rust cores compile their `info` and `debug` logging out of release builds through `release_max_level_warn`, which is good practice. The Kotlin layer has no equivalent: the ProGuard rules contain no `-assumenosideeffects` for `android.util.Log`, so release builds keep `Log.i` lines that print the API base list, the relay gateway, the split-tunnel app list and, on failed cashier calls, URLs carrying the cid (`ConfigManager.kt:274, 416`, `TunnelVpnService.kt:161–165`, `CashierApi.kt:256, 264`). On Android 4.1 and later other apps cannot read these logs, but bug reports and the debug export can. The tun bridge is configured to log at `debug` to a file in every build; what it records could not be checked without its binary. Commits dcf687c, 46ee131 and 8709f28 now exclude preferences, files and databases from both backup paths, keep the bridge at `warn` without a log file in release, and limit the FileProvider to the debug directory.

Network security configuration and cleartext. Release builds disable cleartext traffic through a manifest placeholder and debug builds enable it (`app/build.gradle.kts:23, 88`). There is no network security configuration and no pinning anywhere. For the OTA configuration this is acceptable, because its integrity comes from the Ed25519 signature rather than TLS. For the cashier, the unsigned WGTCP profile and the payment WebView in the `direct` flavour, platform trust is the only protection, and on API 23 devices that includes user-installed CAs. Red Team Ltd recommends a network security configuration that trusts system CAs only, with SPKI pins for the cashier host shipped in, and rotatable through, the signed configuration.

The VpnService and the tun. The interface itself is built well. Routes cover `0.0.0.0/0` and `::/0`. IPv6 is captured with a `fd00::2/128` address rather than left to leak. DNS points at `198.18.0.2` inside the tun, where the bridge is configured to answer with addresses from `100.64.0.0/10` and pass the original names to the core, so that

resolution happens at the exit (the configuration was verified; the bridge binary was not available). The builder never calls `allowBypass()`. The app excludes only itself, as its carrier sockets require, and a bypass list the platform refuses falls back to a full tunnel rather than to none. All of this was confirmed on the emulator (A.20). The app also fails closed when the exit disappears: the watchdog noticed within two intervals, the reconnect failed, the UI said "Connection Failed", and the interface stayed up with its routes, so traffic was blocked rather than leaked (A.32). It survived a change from Wi-Fi to mobile data with the tunnel intact (A.31).

The lifecycle is where it broke at f60abd3, and 5c0d00a repairs most of it. The view model never learned that the platform had taken the tunnel away. After a revocation the UI said "VPN On" more than two minutes later while other apps' traffic left through Wi-Fi, because both of the watchdog's liveness signals (bytes on the app's own UID, and a probe to its own loopback port) are blind to the tun (RVPN-01-310). Always-on VPN starts the service, but the service ignores the system's start action, so always-on yields no tunnel and lockdown yields a dead network after every boot (RVPN-01-312). The app has no in-app kill switch of its own, so lockdown is the only one, and today it cannot be used in a working configuration. The third lifecycle issue is architectural. The tun bridge is pointed at port 1080 before the core binds, and the core falls back silently to another port when 1080 is taken, so any app that occupies 1080 first receives the tunnelled flows while every health check passes (RVPN-01-305). The test bridge stand-in performed hev's first per-flow step against that port and was answered by a listener owned by another UID. Together these were the most important findings of this work package, because each turned the app's own status display into a false statement about protection. Commits 3c82d4f and 264b167 fixed the port takeover and the revocation display; the always-on gap remains. A new race in replacing the tun during a reconnect left the VPN network without routes once in sixteen reconnects (RVPN-01-319).

Loopback architecture. Running the transport as a loopback SOCKS5 server and bridging the tun into it is a reasonable design. It keeps the Rust core free of a TCP/IP stack and makes the core usable from a desktop CLI, which helped this assessment. The design's assumption is that the loopback port belongs to the app. On Android it does not: loopback is shared by all UIDs, the server accepts any client without authentication, and nothing verifies that the port the bridge and the cashier client connect to is the one the core holds (RVPN-01-305, RVPN-01-306). Binding first, handing the bound port to every consumer, and adding per-session SOCKS5 credentials removes both problems without changing the wire protocol. The first two are done in 3c82d4f; the credentials are still missing.

OTA configuration and bootstrap. The signing scheme is the strongest control in the client. Verification happens before parsing, a refused document keeps the last good one, and tampering was refused in the test. The surrounding protocol is incomplete. The signed body carries no sequence, expiry, brand or slot, so any document ever signed can be replayed indefinitely; one forged 404 pins the client to the v1 slot for good; and a single answering base stops all fallbacks (RVPN-01-314). The "sealing" encryption gives no confidentiality, because its key is derived from constants in the client, and the project should not describe it as protecting the endpoint list. The bootstrap is also loud. Every launch fetches the configuration directly, with the product's host name in SNI and the API key in the request, before any tunnel exists. On first run the app registers the cid with the advertising ID while the welcome screen is still showing (RVPN-01-307). The cashier client already defers its calls when a VPN profile exists and the tunnel is down; extending that gate to the configuration fetch, and moving both inside the tunnel, removes the per-launch beacon.

Relay and consent. The consent default was fixed in f60abd3, and the fix was verified in code and on a fresh install (RVPN-01-308). The relay itself is off in the shipped configuration and gated twice, by the signed configuration and by the user's switch, with sensible budgets. If it is ever enabled, it trusts an unauthenticated gateway reached through whichever exit is current and tags the session with the advertising ID; and its consent text describes "anonymous connectivity probes" rather than relaying third-party connections from the user's address (RVPN-01-317). Red Team Ltd recommends that the relay stay disabled until the gateway is authenticated against a key in the signed configuration, the advertising ID is removed from the hello, and the consent screen names the behaviour.

Identity and privacy. The cid is derived from ANDROID_ID with a public salt. It survives reinstallation, cannot be reset, and reaches every exit with the timezone in every carrier. The cashier receives it with the advertising ID, and it is set as the Play account identifier, so the operator can join usage, payments and advertising identity (RVPN-01-309). Endpoint order and the decoy `Host` are also derived from it, so a device looks the same on every session. For a product whose users fear correlation, a random per-install identifier with a reset button,

short-lived exit tokens, and dropping the advertising ID would remove most of this exposure. The advertising ID was removed in 34a5a11 and restored in 49ac756, because the development team uses it to detect fraud and click farms; the cid no longer appears in failure logs (fa567f1); the derived cid and the timezone still reach every exit. The project should also align the privacy policy and the payment copy with what the app sends.

Debug export. A user-initiated diagnostics feature is appropriate for this product. Its encryption is not: the report is encrypted with a symmetric key that ships in the client (all zeros in this tree), so anyone holding the APK can read every report a user forwards (RVPN-01-313). Encrypting to an operator public key fixes this without changing the user's workflow.

JNI boundary and memory safety. The JNI surface binds completely: 29 of 29 declarations in `RedSocketBridge` and 4 of 4 in `WgtcpBridge` resolve, so no call can fail with `UnsatisfiedLinkError` at run time. The Rust code is clean. Clippy reported no warnings in any of the three crates and 223 unit tests passed. `unsafe` appears only in FFI and JNI glue (19 uses in the RedSocket core, 6 in the WGTCP core, none in the vendored client library), and the uses reviewed were sound; for example, the class reference cached in `JNI_OnLoad` is deliberately leaked so that the later raw dereference stays valid (`lib.rs:339–342, 385`). Strings crossing JNI are converted with `jni` 0.21 safe APIs, and network parsers check lengths before slicing: SOCKS5 requests, `u16`-framed Noise messages, the PNG chunk walker, the 256 KB mirror bound and the `.rostam` import bound. Ten malformed SOCKS5 inputs did not disturb the core (A.12). The gaps are hygiene. Four unused JNI exports enlarge the static signature that the project otherwise works to neutralise. Most exported functions lack `catch_unwind`, so any future panic in them aborts the VPN process. And the Kotlin bridge catches `Exception` where an `Error` can occur (RVPN-01-302). MUX2 carriers keep yamux's default 1 GiB connection receive window, while the relay correctly sets its own bound (`mux.rs:217–218` against `peer.rs:117, 425`). A per-carrier bound of a few megabytes would cap what a hostile exit can make the process buffer. This was not exploited and is listed as hardening.

Transport fingerprintability. The HTTP Upgrade carrier is the item with the widest reach (RVPN-01-304). It looks like WebSocket to a human, but to a classifier it is a request with three fixed headers, the fleet-wide path `/noise`, a `Host` that does not match the destination, and no WebSocket key, followed by a constant 2-byte segment and a 48-byte message. Noise NK is used correctly as far as revision 34 of the framework is concerned. The initiator knows and pins the responder's static key, the first handshake message carries an empty payload, and the cid travels only in transport messages. The handshake binds no prologue, so the preceding HTTP exchange is not authenticated. That is harmless while the HTTP fields are decorative, but it should be revisited if they ever carry meaning. The configuration format carries one static key for all endpoints (`config.rs:500–504`); how a production key would be rotated is an open question (Appendix D). Coalescing the frame writes was a client-only change and, in a86a9bb, removed the 2-byte segments. The development team's remediation for the rest of the HTTP signature is the Proteus transport (WP2), with the HTTP carrier kept for legacy clients. Publishing `upgrade_paths` is a configuration change. A real WebSocket or TLS envelope and handshake padding need a coordinated server release and should be planned with WP1. The WGTCP carrier, dormant today, would be blockable on its first record and would leak every destination name to the local resolver. It should not be enabled in censored regions until RVPN-01-317 is addressed.

The hev-socks5-tunnel dependency. The component that sees every packet is not in the repository, is not pinned to a version, commit or digest, and is listed in the app as GPL-3.0 inside an Apache-2.0 project, with no licence text or source offer in the tree (RVPN-01-318). Its absence limited this assessment: the bridge's logging, its IPv6 handling (its configuration names only an IPv4 address) and its behaviour under load could not be tested. Vendoring it at a pinned commit, with a build script, would make it reviewable and would settle the licence question.

Supply chain and regulation. `cargo audit` found no vulnerable crate in 171, 196 and 172 locked dependencies. The single yanked crate, `chacha20` 0.10.1, is reached only through `rand` 0.10.2 in the vendored crate's CLI binary and should be updated. Everything Rust is vendored and resolved offline when built as intended. The project produces no SBOM and no build provenance, publishes no security policy, and has no `SECURITY.md`. If the paid app is made available in the European Union, the Cyber Resilience Act's reporting duties under Article 14 have applied since 11 September 2026, and the remaining obligations, including documented component inventories, apply from 11 December 2027. A CycloneDX 1.7 or SPDX 3.0.1 SBOM, SLSA provenance for release builds and a published disclosure contact would cover the practical ground. NIST SP 800-218 practices PS.3 and PW.4 describe the same expectations.

Standards conformance. Against OWASP MASVS v2.1.0 the app passes the cryptography and code-quality controls that matter most for a VPN client. At f60abd3 it failed STORAGE-2, NETWORK-1, NETWORK-2, PLATFORM-1, PLATFORM-3 and PRIVACY-1; after the remediation it fails NETWORK-1 and NETWORK-2, passes PLATFORM-3 and partially meets the rest (Appendix C, which records the state at 5c0d00a). The resilience controls were judged not applicable to an open-source circumvention tool, and the project has rightly not invested in obfuscation. Protocol-level non-conformance was found against RFC 6455 (the upgrade is not a WebSocket handshake) and RFC 1928 §3 (method selection). The use of `Upgrade: websocket` without WebSocket semantics also departs from RFC 9110 §7.8.

Re-verification of the earlier static review. Every hypothesis in the earlier review was checked against f60abd3. Confirmed and carried over, most of them now with dynamic evidence: the carrier fingerprint, the launch-time requests, the decoupling of UI state from the tun (revocation), the always-on gap (previously only suspected), the absence of configuration freshness and the sticky v1 downgrade, the backup scope, the lock-screen display, the account-code deep link and protocol extra, the stable identifiers, the client-held debug-report key, the unguarded JNI entry points, the dormant control-stream, relay and WGTCP issues, and the supply-chain gaps. Corrected: a forced reconnect leaves no multi-second unprotected window (19 ms measured); the cid appears in logcat only when a cashier request fails, which did not happen during testing; and the claim that the app carries no telemetry SDK does not hold, because the Play Billing Library queues telemetry events. Resolved by f60abd3: the relay consent default, the exit addresses, the personal mailbox and the flagged operational note; resolved by b69874a: the remaining CDN host names. New in this assessment: the port-1080 takeover, the 2-byte frame segmentation, the failing build instructions and embedded build paths, the unused JNI exports, and the structure of the publisher-sealed configuration.

Verdict. The client is not compromised by any remote, unauthenticated attack that Red Team Ltd could find, and its cryptography, configuration signing and parsers are solid. At f60abd3 it could not be relied on to tell its user the truth about protection. After the remediation pass it largely can: the port takeover and the revocation display are fixed, and registration no longer leaves the device before a tunnel. What remains are an always-on start that does nothing, a tun replacement that can leave the VPN network without routes, a carrier whose HTTP signature awaits the Proteus transport, a configuration fetch that still names the product at every launch, and the open privacy, debug-export and supply-chain items listed below.

Prioritised recommendations.

1. Replace the tun without a teardown race: keep the interface across reconnects, or establish the replacement before closing the old descriptor on the same thread, and confirm the VPN network's routes after `establish()` (RVPN-01-319).
2. Support the always-on start, or declare it unsupported, so that lockdown becomes a working kill switch (RVPN-01-312).
3. Add per-session SOCKS5 credentials shared with the bridge, and answer `05 FF` when no acceptable method is offered (RVPN-01-306).
4. Ship the Proteus transport as planned, and publish `upgrade_paths` for the HTTP carrier that legacy clients keep (RVPN-01-304).
5. Encrypt debug reports to an operator public key (RVPN-01-313).
6. Add sequence, expiry, brand and slot to the signed configuration, and fix the fallback and downgrade logic (RVPN-01-314).
7. Replace the SSAID-derived cid with a resettable random identifier, keep it out of the exit hello, and update the policy and the Play Console declaration (RVPN-01-309); give the launch-time configuration fetch a non-product SNI (RVPN-01-307).
8. Deliver account codes through verified App Links (RVPN-01-311).
9. Before enabling any dormant feature, sign control frames and the blocklist, authenticate the relay gateway, and resolve WGTCP names inside the tunnel (RVPN-01-317).
10. Enable Gradle dependency verification, vendor and pin the tun bridge, remap the full build paths, guard the JNI entry points, and publish an SBOM and a security policy (RVPN-01-318, RVPN-01-303, RVPN-01-302).

Red Team Ltd thanks the RostamVPN maintainers for a well-structured public tree, an offline build that worked on the first attempt, and code whose comments explain intent in a way that made a white-box review of this depth possible within the window.

Appendix A: Dynamic test log

Environment and harness disclosure. Assessment host: macOS 26.6.2 on Apple Silicon. Scratch directory `/tmp/rvpn01/wp3`; the repository was copied to `/tmp/rvpn01/wp3/tree` (excluding `.git` and the pre-existing `redsocket-http-core/target`) and never written to. Harness services bound to 127.0.0.1 only: test exit 20310, core CLI SOCKS port 20301, cashier stub 20311, probe origin 20312. The Android Emulator and adb necessarily used ports outside the WP3 block (console 5554, adb 5555 and 5037, gRPC 8554 and one ephemeral port); the emulator's `-port` option accepts only 5554 to 5682, so this is recorded as a tool constraint. Two APKs were built. APK1 was built from the unmodified tree plus the two cores (sha256 `6cce02cddddc8d92a10cf3dce37bd38d6ec1080d84a8221f35a64b94f0d59e3e`). It contained the first, non-LTO core build and was used only for packaging checks; it was not run. APK2 (sha256 `e8fc5f628a4b18d3e425d740ab40bff1f4a6ab1a49cc47f6fa5148c3710e00e5`) contained both cores built with their own release profiles (fat LTO), the hev stub, and the scratch copy's `default-config.json` edited as described in the methodology; its packaged configuration was checked with `unzip -p`. The AVD `pingdyn30` carried data from an earlier installation of the package, including a cached server configuration. The first cold-launch run therefore read that cache, and no request reached the stub because guest DNS was isolated. The run was discarded, the package was uninstalled inside the session overlay, and the test was repeated on clean data; no value from that cache is reproduced here. Processes started and stopped by PID: test exit 82997, 7772 and 66838; core CLI 83055, 83911 and 66843; tcpdump 83902; probe origin 82994; cashier stub 88526; emulator 88700; adb server 86501; device-side listener 5909 and tun poller 6840 (killed with `adb shell kill`). After testing no listener remained in the block or on the emulator ports, and the AVD's `userdata-qemu.img.qcow2` modification time was unchanged.

A.1 Working tree against git index. Host. `python3 -I index_check.py <repository>` (recomputes `sha1("blob <len>\0" + content)` for every index entry). Pass.

```
index v2: 22843 entries; match=22843 mismatch=0 missing=0
HEAD: ref: refs/heads/main -> f60abd3722a93ff9d7b0b4ad5e0a99fac688c135
```

A.2 Offline debug APK build. Host, JDK 17.0.15, `GRADLE_USER_HOME=/tmp/rvpn01/wp3/gradle-home` holding only the cached 8.14.4 distribution. Time box started 19:04:12. Pass.

```
$ bash ./gradlew --offline --no-daemon -Pkotlin.compiler.execution.strategy=in-process \
   --console=plain :app:assembleRostamPlayDebug
BUILD SUCCESSFUL in 1m 39 actionable tasks: 39 executed (end 19:05:35)
rebuild with harness config and stub: BUILD SUCCESSFUL in 6s; sha256 changed as above
```

A.3 Native builds. Host, NDK 28.0.13004108, run from each crate directory. Pass.

```
redsocket-http-core: cargo ndk -t arm64-v8a -o <jniLibs> build --release --offline --locked
  (first run with CARGO_PROFILE_RELEASE_LTO=false: 9.09 s; release profile: 19.14 s)
wgtcp-android: same command, release profile: 16.67 s
```

A.4 README build command. Host, scratch tree root. Fail (RVPN-01-303).

```
$ cargo ndk -t arm64-v8a -o $JNI --manifest-path redsocket-http-core/Cargo.toml build --release --offline
error: `cargo metadata` exited with an error: error: could not find `Cargo.toml` in ...
$ cargo metadata --manifest-path redsocket-http-core/Cargo.toml --format-version 1 --offline
  from the root: {'registry': 148, ...} from the crate directory: {'vendor': 148, ...}
```

A.5 JNI surface. Host. `llvm-nm --defined-only -D` on both libraries, compared with `grep 'external fun'`. 29/29 and 4/4 bound; five extra exports (RVPN-01-302).

A.6 Embedded paths. Host. `strings -a libredsocket_core.so | grep -c '^/build/'` returned 32, and 20 for `libwgtcp_core.so`; no unremapped home-directory path was present. Fail (RVPN-01-303).

A.7 Unit tests. Host, `cargo test --offline --locked` per crate, separate target directories. Pass.

```
redsocket-http-core: 136 passed; 0 failed; 6 ignored (live tests, not run)
wgtcp-android: 3 passed vendor/redsocket_wgtcp: 82 passed + 2 passed
```

A.8 Lints. Host, `cargo clippy --offline --locked --all-targets`. Pass, 0 warnings in each crate.

A.9 Advisories. Host, `cargo audit --no-fetch --stale`, database of 1,329 advisories fetched 2026-10-14 17:55. 0 vulnerabilities; `chacha20` 0.10.1 yanked in `wgtcp-android` and `vendor/redsocket_wgtcp`.

A.10 Yanked crate trace. Host, `cargo tree --offline --locked --target aarch64-linux-android -i chacha20@0.10.1`: `rand v0.10.2 <- redsocket-wgtcp v0.1.0 <- wgtcp-android v1.0.0`.

A.11 Carrier capture. Host. Test exit with a fresh key pair; core CLI (`--features cli`) with a scratch configuration whose only endpoint is 127.0.0.1:20310; `tcpdump -i lo0 -w carrier-mux.pcap 'tcp port 20310'`. Fail (RVPN-01-304); output in that item.

A.12 SOCKS5 malformed input. Host. `python3 -I socks_malformed.py 20301`, ten cases, each followed by a full CONNECT through the core to the probe origin. Pass for robustness; RFC 1928 §3 deviation (RVPN-01-306).

```
empty then close -> <timeout> | SOCKS4 version byte -> (closed) | nmethods=255 -> (closed)
only user/pass offered -> 0500 | domain len 255 truncated -> 0500 | ATYP 0x09 -> 0500
IPv6 truncated -> 0500 | BIND -> 0500 0507... | 512 random bytes -> (closed)
CONNECT to a port outside the block -> 0500 0500... (the test exit refused both the MUX2
  attempt and the legacy fallback; the legacy path had already answered the client with success)
still healthy after each case: True ; CLI alive: yes
```

A.13 Sealed vector structure. Host. Scratch crate depending on `redsocket-http-core` by path, with its own `.cargo/config.toml` pointing at the core's vendor directory; `cargo run --offline -- <cert hash from cfgseal.rs:167>`. Fail (RVPN-01-314); output in that item.

A.14 Identifier sweep. Host. `python3 -I ident_sweep.py <repository>` over non-vendored text files. Fail (RVPN-01-301); classes and locations in that item.

A.15 Emulator boot and isolation. `emulator -avd pingdyn30 -no-window -no-audio -no-boot-anim -no-snapshot -read-only -netfast -dns-server 127.0.0.1 -gpu swiftshader_indirect`. Boot completed in 27 s; `ping -c1 example.org` in the guest returned `unknown host`. Pass.

A.16 Package inspection. `adb install` of APK2, then `dumpsys package com.rostamvpn.android`: flags `DEBUGGABLE HAS_CODE ALLOW_CLEAR_USER_DATA ALLOW_BACKUP`; requested permissions `AD_ID`, `INTERNET`, `FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_SPECIAL_USE`, `ACCESS_NETWORK_STATE`, `BILLING`; resolver entries only for `.MainActivity`, `.TunnelVpnService` (with `BIND_VPN_SERVICE`) and the debug-only `.DebugReceiver`. Informational.

A.17 Cold launch. Clean data; `am start -W -n com.rostamvpn.android/.MainActivity`, wait 15 s, cashier stub log. Fail (RVPN-01-307).

A.18 Resume. HOME, then `am start` again; cashier stub log and `[PAY] register ok tier=free ifa=yes lmt=0` in logcat. Fail (RVPN-01-307).

A.19 Relay switch. Settings opened through the UI; `uiautomator dump`; `run-as ... cat shared_prefs/rostam_prefs.xml | grep -c probe_enabled` returned 0. Pass (RVPN-01-308).

A.20 Connect and tun inspection. Connect through the UI; VPN consent dialog accepted. Pass.

```
tun0: inet 10.0.0.2/32, inet6 fd00::2/128, mtu 1500
LinkProperties: DnsAddresses: [ /198.18.0.2 ]
Routes: [ 0.0.0.0/0 -> 0.0.0.0 tun0, ::/0 -> :: tun0, 10.0.0.2/32, fd00::2/128 ]
/proc/net/tcp: LISTEN 127.0.0.1:1080 uid=10183 (app)
hev-socks5-tunnel.yml: socks5 127.0.0.1:1080, mapdns 198.18.0.2:53, network 100.64.0.0/10, log-level
    -> debug
test exit: GET /noise HTTP/1.1, Host: teams.microsoft.com (91 bytes); hello META|America/Los_Angeles|||00
    -> f70560...
```

A.21 Competing listener on 1080. Shell-UID listener started with `nc -4 -L -s 127.0.0.1 -p 1080`, then Connect. Fail (RVPN-01-305); output in that item.

A.22 Revocation. System Settings, VPN, RostamVPN, Disconnect at 19:26:17; state read immediately and again at 19:28:38. Fail (RVPN-01-310); output in that item.

A.23 External select_protocol. On-device poller `while true; do [ -e /sys/class/net/tun0 ] ...; sleep 0.05; done`, then `am start ... --es select_protocol wgtcp` from the shell. 262 samples, no down interval detected;

logcat places close and re-establish 19 ms apart. Fail for the unauthenticated extra, not for a leak window (RVPN-01-311).

A.24 Always-on and lockdown. Settings toggles; `settings get secure always_on_vpn_app`; `dumpsys activity services`; `ip route get` and `nc` from the shell; then lockdown cleared with `settings put secure always_on_vpn_lockdown 0` and always-on switched off in Settings (both switches read off; `always_on_vpn_app` returned `null`). Fail (RVPN-01-312).

A.25 Account-code deep link. With the tunnel down: `CashierApi: POST .../auth/claim failed: cashier deferred`. With the tunnel up: claim posted (output in RVPN-01-311). An implicit intent opened the system chooser. Fail.

A.26 Cold relaunch with a VPN profile. `am force-stop`, `am start -W`, HOME and resume. Only `GET /api/v1/config` (direct, with `X-Api-Key`) reached the stub; registration deferred; `config_no_v2` was `true`. Fail (RVPN-01-307, RVPN-01-314).

A.27 File URI. `am start ... -a android.intent.action.VIEW -d file:///data/data/com.rostamvpn.android/shared_prefs/redsocket.xml`. Logcat recorded the first 40 characters of the file; the toast read "not a config code". Fail (RVPN-01-311).

A.28 Debug report. Settings, Share Debug Log, share sheet dismissed; `run-as ... cat cache/rostam-debug-*.enc`; decrypted on the host with `java Decrypt.java` using a 32-byte zero key. Two exports were made; in both the logcat section was empty. Fail (RVPN-01-313).

A.29 Backup. `bmgr transport com.android.localtransport/.LocalTransport`, `bmgr enable true`, `bmgr backupnow com.rostamvpn.android`, archive read as root from `/data/data/com.android.localtransport/files/1/_full/` and listed with `tar -tvf`; datatransport database inspected with `sqlite3`. Fail (RVPN-01-315).

A.30 Lock screen. `locksettings set-pin 1234`, app in foreground, `KEYCODE_SLEEP`, `KEYCODE_WAKEUP`, `dumpsys window`, `dumpsys activity activities`, Settings opened while locked, `locksettings clear --old 1234`. Fail (RVPN-01-316).

A.31 Network change. Connected; `svc wifi disable`; after 20 s the UI showed "VPN On" and one VPN network remained; `svc wifi enable`; one VPN network. No crash. Pass.

A.32 Exit loss. Connected; test exit stopped by PID at 21:54:56; state read at 21:57:26. Pass, fail-closed.

```
[HEALTH] probe 1/2 failed ... probe 2/2 failed ... tunnel is dead, auto-reconnect 1/3
[RS] Failed: no http endpoint reachable ; VPN interface established, fd=68
UI: "Connection Failed", "Tap to connect" ; VPN networks: 1
ip route get 192.0.2.1 uid 10140 -> dev tun1 (traffic held in the tun, not leaked)
```

A.33 Repository unchanged. `find <repository> -newer timebox-start.txt -not -path './.git/*'` returned nothing before this report was written. Pass.

A.34 Fix verification, commit b69874a. Host, no git. `.git/refs/heads/main` read `b69874ae40a34e9367df7f4e35be654f8c9ab987`; the loose commit object was decompressed with Python `zlib` and its tree walked along the changed paths; `diff -rq` against the f60abd3 snapshot in `/tmp/rvpn01/wp3/tree`; `ident_sweep.py` re-run; `cargo check --offline --locked` in a fresh scratch copy of the current tree. Pass for RVPN-01-301.

```
commit b69874ae40a3: parent f60abd3722a9..., "Replace the remaining CDN front hostnames with placeholders
    "
brands/configs/rostam.json / default-config.json: commit blob = working tree (MATCH)
changed lines: rostam.json:9, 141-143, 320-322; default-config.json:9 -> placeholders
also changed: wgtcp-android/vendor/smoltcp/src/iface/rpl/trickle.rs and the same file under
  vendor/redsocket_wgtcp/vendor/ (a third-party documentation link)
sweep: no CloudFront or Fastly host name outside placeholders and test values
cargo check, wgtcp-android: error: the listed checksum of
  `.../vendor/smoltcp/src/iface/rpl/trickle.rs` has changed
cargo check, redsocket-http-core: Finished
```

Version 1.1 re-verification: environment and harness. On 2026-10-15 the repository at 5c0d00a was copied to `/tmp/rvpn01/wp3/tree5`, outside the repository, and both cores were built with their release profiles. Two debug APKs were built offline. APK5A, from the unmodified tree plus the cores, has sha256 `a74906fa51c562f8d59eb77491d77e91a2461dfd090e68f5355eb8b77ccc81bb` and was used for manifest and packaging

checks. APK5B, with the same loopback configuration edits and bridge stand-in as before, has sha256 `5d6d1d45e7b6e1377fe8b349cbaaaeef6b10479cd6232039a96afef8e61d0d53` and was used on the emulator. The emulator ran with the same flags as in A.15. The AVD again carried the earlier installation, which was uninstalled inside the session overlay before testing. Processes started and stopped by PID: probe origin 8992; test exit 8995 and 25974; tcpdump 8998; core CLI 9008 and 23250; cashier stub 9257, 15405, 16983 and 17975; emulator 9260; adb server 9267; device-side listener 5375. Afterwards no listener remained in the block or on the emulator ports, the AVD image's modification time was unchanged, and the scratch build trees were deleted. One file of supporting evidence came from the development team's own run under `/tmp/rvpn01/fix-wp3` (the `adb backup` archive in A.51); every other result below is Red Team Ltd's own.

Four incidents are recorded. (1) A first always-on attempt was made while the VPN was connected; it exercised nothing and was discarded. (2) On the second attempt the Settings list order had changed, and always-on was enabled for another VPN app installed on the AVD, a separate build of the same code base, for about ten seconds. It was switched off at once and that app's process was force-stopped inside the guest; guest DNS was isolated throughout and no VPN network came up. The test was then repeated for RostamVPN, with the row selected by its title. (3) After the lock-screen test the PIN bouncer stayed up although the credential had been cleared, and the guest was rebooted inside the session overlay. (4) The first host run of the core tests reported 8 failures, because the harness had rewritten the scratch copy's bundled configuration, which those tests read with `include_str!`; with the original file restored, all tests passed (A.41).

A.35 Commit chain and diffs. Host, no git. `python3 -I git_chain.py <repo> <HEAD> b69874a`, then `gitread.py` (loose and pack-v2 objects with delta resolution) comparing the trees of f60abd3 and 5c0d00a. Twenty commits after b69874a (621ba38 and nineteen remediation commits), plus 82623d9; 40 first-party or library files changed, and the vendored dependencies of `vendor/redsocket_wgtcp` were pruned. Each changed file was attributed to its commit.

A.36 Working tree against git index at 82623d9. `python3 -I index_check.py <repo>: index v2: 22714 entries; match=22714 mismatch=0 missing=0`. Pass.

A.37 Native builds at 5c0d00a. `cargo ndk -t arm64-v8a ... build --release --offline --locked` in both crate directories. The core finished in 11 s; the WGTCP core also built, so the b69874a regression is resolved by 621ba38. Pass.

A.38 JNI surface and embedded paths. `llvm-nm --defined-only -D` against the Kotlin declarations: 29/29 bound plus `clearBlockHistory`, `getBlockStats`, `setBlocklist` and `setUserRule`; 4/4 plus `nativeGetConnectionStatus`. `strings -a ... | grep -c '^/build/'` returned 32 and 20. Unchanged.

A.39 Debug APK builds and merged manifest. `aapt2 dump permissions` on APK5A lists INTERNET, FOREGROUND_SERVICE, FOREGROUND_SERVICE_SPECIAL_USE, ACCESS_NETWORK_STATE and BILLING, with no `AD_ID`; the activity's data schemes are `rostam` and `content`; `allowBackup=true` with the new exclusion rules. Pass; superseded by A.58.

A.40 Carrier capture. Test exit and core CLI rebuilt; `tcpdump -i lo0 'tcp port 20310'`. The MUX2 carrier's client segments were 91 (request), 50, 43, 42, 32, 38, 108, 30; `client 2-byte segments: 0 of 11`. The request was still `GET /noise HTTP/1.1` with `Host: graph.microsoft.com`. Partial (RVPN-01-304).

A.41 Rust suites at 5c0d00a. `cargo test --offline --locked`: core 137 passed, 0 failed, 6 ignored (including `a_frame_is_one_write_of_the_same_bytes`); `wgtcp-android` 3 passed; `vendor/redsocket_wgtcp` 100 and 2 passed. `cargo clippy --all-targets`: 0 warnings in each. `cargo audit --no-fetch --stale`: 0 vulnerabilities in 171, 173 and 166 dependencies, and no yanked crate. Pass.

A.42 App JVM unit tests. `./gradlew --offline :app:testRostamPlayDebugUnitTest`: HevConfigTest 3, IpRoutingTest 27, PngPayloadTest 7, CashierRouteTest 2; 0 failures. Pass.

A.43 SOCKS5 malformed input. `python3 -I socks_malformed.py 20301` against the rebuilt CLI: same ten results as A.12; the core still answered `0500` to a client offering only username/password. RVPN-01-306 open.

A.44 Cold launch and resume. Fresh install of APK5B; launch, HOME, resume. The cashier stub saw only `GET /api/v2/config` and `GET /api/v1/config`, both with `X-Api-Key`; logcat showed `CashierApi: POST .../cid/register failed: cashier deferred: tunnel down`. Partial (RVPN-01-307).

A.45 Connect sequence. Logcat: `VPN interface established` then `VPN started; the bridge waits for the core's port, [RS] verified, [PAY] tunnel up and cid unregistered, retrying through it, [PAY] register ok tier=free`, and only then `Starting hev-socks5-tunnel ... port=1080`. The stub recorded `{"cid": "00f70560...", "brand": "rostam", "tz": "America/Los_Angeles"}` with no `ifa`, reached through `MUX2 CONNECT 10.0.2.2:20311` on the test exit; the test exit decrypted the hello `MUX2|America/Los_Angeles|||00f70560...|`. Pass.

A.46 Competing listener on 127.0.0.1:1080. Same listener as A.21 (device PID 5375), then Connect. Pass (RVPN-01-305).

```
RedSocket: [socks5] 127.0.0.1:1080 is taken by something else; using 44713 instead
VpnViewModel: [RS] verified (carrier hidden from UI)
RostamVPN: Starting hev-socks5-tunnel via JNI with fd=64, port=44713, udp=false
RVPN01-HEV-STUB: greeting to 127.0.0.1:44713: sent 05 01 00, reply n=2 05 00
squat.log: (empty) /proc/net/tcp: 127.0.0.1:44713 uid=10183, 127.0.0.1:1080 uid=2000
```

A.47 Revocation. System Settings, VPN, RostamVPN, Disconnect at 00:13:36: `[VPN] tun revoked or not established, disconnecting`; UI "VPN Off", "Tap to connect"; VPN networks 0; no listener left. Pass (RVPN-01-310).

A.48 External extras and URIs. `am start ... --es select_protocol wgtcp --ez request_vpn true` produced no reconnect, and `selected_protocol` stayed unset. `file:///data/data/com.rostamvpn.android/shared_prefs/redsocket.xml` and `content://com.rostamvpn.android.fileprovider/debug/x.rostam` each produced `Config import refused: not another app's content URI`. Pass.

A.49 Account-code link. `am start ... -d 'rostam://c?code=RVPN01-TEST-CODE'` opened a dialog showing the code with Cancel and Restore. Cancel: no `/auth/claim` request. Restore: `POST /api/v1/auth/claim {"code": "RVPN01-TEST-CODE", "cid": "<cid>"}`. Pass.

A.50 Always-on. From a disconnected state: `always_on_vpn_app=com.rostamvpn.android`, service started with `intent={act=android.net.VpnService`, VPN networks 0; switched off again (`null`, lockdown 0). See incidents (1) and (2). Fail (RVPN-01-312).

A.51 Backup. `bmgr backupnow com.rostamvpn.android` over the local transport: `Transport rejected package because it wasn't able to process it at the time`, which the local transport returns for a zero-size full backup; nothing was stored. The development team's `adb backup` archive (`backup-after.ab`) decompresses to a tar holding only `apps/com.rostamvpn.android/_manifest`. Pass (RVPN-01-315).

A.52 Lock screen. PIN set, app in front, sleep and wake: `mCurrentFocus=Window{... NotificationShade}`, `mKeyguardShowing=true`, no element of the app on screen. Pass (RVPN-01-316).

A.53 Cashier failure logs. Connected; the cashier stub restarted with an option that drops `GET /api/v1/usage`. Logcat: `CashierApi: GET http://10.0.2.2:20311/api/v1/usage failed: unexpected end of stream ...`, without the query string; cid occurrences in logcat 0; the requests themselves still carried `?cid=`. Pass.

A.54 Debug report. Share Debug Log while connected: `[debug] diag post failed: javax.net.ssl.SSLHandshakeException: connection closed`, because the post went through the tunnel and the test exit refused `diag.diag.example.net:443`. `cache/debug/rostam-debug-20261015-002144.enc` (767 bytes) decrypted with the zero key to model, endpoints and timezone; the logcat section was empty. Fail (RVPN-01-313).

A.55 Exit lost while connected. Test exit stopped by PID at 00:24:47. The watchdog missed twice and reconnected at 00:26:23; the reconnect failed and the UI showed "Connection Failed". VPN networks 1; `ip rule` pointed at table 1016, which was empty; `ip route get 192.0.2.1 uid 10140` went via `wlan0`. Fail (RVPN-01-319); output in that item.

A.56 Commit 82623d9. Loose-object diff: `wgtcp-android/src/lib.rs` only, where `status()` returns `reconnecting` while `RUNNING` is set and `carrier_is_up()` is false (lines 56-60). It is reachable through the C ABI `wgtcp_status` (`ffi.rs:83-84`) and the undeclared JNI export. With the file applied to the scratch tree: `cargo test` 3 passed and clippy reported 0 warnings. Pass.

A.57 Reconnect series. `reconnect_loop.sh 8`: eight split-tunnel reconnects, each toggling Calendar in exclude mode. `reconnect_fresh.sh 5`: five fresh connects, each followed by one reconnect. One connect from the failed state with the exit restarted (00:28:49). A second exit loss with a fresh tun0, where the watchdog reconnect

failed at 00:40:01. After every one of these 15 reconnects, `ip route get 192.0.2.1 uid 10140` resolved to the current tun interface (`tun0` or `tun1`), and both tun0 and tun1 were sometimes present at once. Logged orders included `stale teardown: gen 3 superseded by 4, tun left alone`. Routing correct in all 15.

A.58 Kotlin build and merged manifest at 49ac756. After the development team restored the advertising ID (49ac756, reversing 34a5a11 and the guard in b793126), `gradlew --offline :app:compileRostamPlayDebugKotlin :app:processRostamPlayDebugMainManifest` against the mirror succeeded. The merged manifest for `rostamPlayDebug` lists `com.google.android.gms.permission.AD_ID` with INTERNET, FOREGROUND_SERVICE, FOREGROUND_SERVICE_SPECIAL_USE, ACCESS_NETWORK_STATE and BILLING. `Gaid.kt`, the `ifa` and `lmt` registration fields and the advertising-ID relay tag are back in the source; registration still goes through the tunnel only (`Route.TUNNEL`). Advertising ID present (RVPN-01-307, -309, -317).

Appendix B: Dependency inventory and advisories

B.1 Rust. All registry dependencies are vendored in each crate's `vendor/` directory and selected through `.cargo/config.toml`; no lock file contains a git source, and every registry package carries a checksum.

Lock file	Packages	Registry with checksum	cargo audit (1,329 advisories)
`redsocket-http-core/Cargo.lock`	171	170	0 vulnerabilities, 0 warnings
`wgtcp-android/Cargo.lock`	196 (173 at 5c0d00a)	194	f60abd3: `chacha20` 0.10.1 yanked; 5c0d00a: clean
`vendor/redsocket_wgtcp/Cargo.lock`	172 (166 at 5c0d00a)	171	f60abd3: `chacha20` 0.10.1 yanked; 5c0d00a: clean

Security-relevant versions. RedSocket core: `snow` 0.9.6 (Noise), `chacha20poly1305` 0.10.1, `ed25519-dalek` 2.2.0, `blake2` 0.10, `yamux` 0.13.10, `tokio` 1.53.1, `jni` 0.21.1, `rand` 0.8.7 and 0.9.5. WGTCP core: `boringtun` 0.7.1, `smoltcp` 0.13.1, `x25519-dalek` 2.0.1, `ring` 0.17.14, `chacha20poly1305` 0.10.1, `rand` 0.10.2. At f60abd3 the yanked `chacha20` 0.10.1 was reached only through `rand` 0.10.2, which the vendored crate used in its CLI binary; the advisory database records no reason for the yank and the registry was not consulted. Commit 5c0d00a removed `rand`, made `tun` optional and builds the client without the `server` feature, which accounts for the smaller lock files.

B.2 Gradle. Build: Gradle 8.14.4 (wrapper without `distributionSha256Sum` at f60abd3, pinned to the value below in 4906b79; the cached distribution archive used for the build had SHA-256 `f1771298a70f6db5a29daf62378c4e18a17fc33c9ba6b1436`, not compared with the published value), Android Gradle Plugin 8.13.2, Kotlin Gradle and Compose compiler plugins 2.3.21. The application declares (`app/build.gradle.kts:215-246`): Compose BOM 2026.06.01 (material3, ui, ui-tooling-preview, foundation, animation, material-icons-core; ui-tooling for debug only), activity-compose 1.11.0, lifecycle-viewmodel-compose and lifecycle-runtime-compose 2.9.4, appcompat 1.8.0, activity-ktx 1.10.1, core-splashscreen 1.2.0, Material Components 1.14.0, Play Billing (billing-ktx) 8.0.0, play-services-ads-identifier 18.3.0 (removed in 34a5a11, restored in 49ac756), zxing core 3.5.4 (declared, not referenced), Play In-App Review 2.0.2; tests: junit 4.13.2, androidx.test.ext junit 1.1.4, espresso-core 3.5.0. No dynamic versions. The mirror holds 475 modules (194 JAR, 89 AAR) with no checksum or signature files, and no `verification-metadata.xml` exists. No CVE scan of the Maven artefacts was performed: no offline vulnerability database for Maven was available on the host, and the rules of engagement excluded fetching one during testing.

B.3 Native components outside Cargo. `libhev-socks5-tunnel.so`: not in the tree; no version, commit or digest; licence stated in the app as GPL-3.0 (`OpenSourceScreen.kt:87-88`); the Java glue `app/src/main/java/hev/htproxy/TProxyService.java` carries no licence header. `libandroidx.graphics.path.so` is packaged by an AndroidX dependency.

B.4 Unpinned or unverified sources. Remote Maven repositories after the mirror (`settings.gradle:4-6`, `build.gradle.kts:7-8, 22-23`); the Gradle wrapper distribution; the tun bridge binary. No SBOM (CycloneDX or SPDX) or SLSA provenance is produced. SwiftPM does not apply to WP3.

Appendix C: Standards conformance checklist (OWASP MASVS v2.1.0)

Results reflect commit 5c0d00a with 82623d9.

Control	Result	Note
MASVS-STORAGE-1	Partial	No credentials stored; WireGuard key in private preferences (not Keystore), excluded from backup; cid and state in plain preferences
MASVS-STORAGE-2	Partial	Backups now exclude app data (RVPN-01-315, fixed); debug report readable with in-app key (RVPN-01-313)
MASVS-CRYPTO-1	Pass	Noise NK with ChaCha20-Poly1305 and BLAKE2s, Ed25519, X25519, OS CSPRNG; no weak primitives found
MASVS-CRYPTO-2	Partial	Debug-report key in the client (RVPN-01-313); one Noise static key per configuration, no documented rotation path
MASVS-AUTH-1	Not assessed	No user accounts; entitlement decided by the cashier (out of scope)
MASVS-AUTH-2	Not assessed	No local authentication in the app
MASVS-AUTH-3	Partial	Code claims now confirmed; codes still delivered by a custom scheme (RVPN-01-311)
MASVS-NETWORK-1	Fail	HTTP signature, direct launch-time fetch, always-on gap, tun replacement race (RVPN-01-304, -307, -312, -319)
MASVS-NETWORK-2	Fail	No pinning or network security configuration; config protected by signature only (RVPN-01-314)
MASVS-PLATFORM-1	Partial	Port takeover fixed (RVPN-01-305); SOCKS5 unauthenticated (RVPN-01-306); intents largely fixed (RVPN-01-311)
MASVS-PLATFORM-2	Partial	One JavaScript WebView for hosted checkout in the `direct` flavour, no URL allow-list, no JS bridge
MASVS-PLATFORM-3	Pass	Lock-screen display and revoke state fixed (RVPN-01-316, -310)
MASVS-CODE-1	Pass	targetSdk 36, current AGP and Kotlin
MASVS-CODE-2	Partial	Release minified and native stripped; build paths embedded (RVPN-01-303)
MASVS-CODE-3	Partial	Rust audited clean, yanked crate removed; Maven not scanned; tun bridge unpinned (RVPN-01-318)

Control	Result	Note
MASVS-CODE-4	Partial	Parsers robust under malformed input; unguarded JNI entry points (RVPN-01-302); no config freshness (RVPN-01-314)
MASVS-RESILIENCE-1	Not assessed	Root and tamper detection deliberately absent in an open-source tool
MASVS-RESILIENCE-2	Not assessed	As above
MASVS-RESILIENCE-3	Not assessed	Source is public; static signature reduction tracked in RVPN-01-302 and -303
MASVS-RESILIENCE-4	Not assessed	Not appropriate for this product
MASVS-PRIVACY-1	Partial	Advertising ID collected again since 49ac756 for fraud detection; derived cid and timezone still sent to every exit (RVPN-01-309)
MASVS-PRIVACY-2	Partial	Stable, reinstall-proof cid in every carrier hello (RVPN-01-309)
MASVS-PRIVACY-3	Partial	Relay consent default fixed (RVPN-01-308); relay copy and policy incomplete (RVPN-01-317, -309)
MASVS-PRIVACY-4	Partial	Relay opt-out exists; no identity reset

Appendix D: Open questions for the maintainers

1. Which upstream commit, build flags and toolchain produce the shipped `libhev-socks5-tunnel.so`, what does it log at `debug`, and under which licence is it distributed?
2. Is the production `api.base` a dedicated host name, and is it fronted? Is ECH or a fronted SNI used for the bootstrap anywhere?
3. Were the three CloudFront distributions and the CDN host name that `brands/configs/rostam.json` carried before b69874a live, and should they be rotated, given that they remain in the public history?
4. How will the Noise static key that a production configuration carries for all endpoints, and the configuration verifying key, be rotated, and is there a second verifying key slot in the client?
5. Has any configuration with `probe.enabled`, `protection`, `messaging` or `wgtcp` been signed for any brand? Such documents are replayable today (RVPN-01-314).
6. What does `/api/v1/auth/claim` do when a code is claimed for a cid that already has an entitlement, and can a code be claimed repeatedly?
7. How does the production exit answer the upgrade request, including to an unknown or replayed request (active-probing resistance; WP1)?
8. Is lockdown mode with always-on intended to be supported, and should the app offer an in-app kill switch?
9. Will release builds strip Kotlin `Log` calls, and is the in-app logcat capture expected to return content on current Android versions?
10. Has a VPN network without routes after a reconnect (RVPN-01-319) been seen on devices, and on which Android versions?